ENGENHARIA DE SOFTWARE PARA INTELIGÊNCIA ARTIFICIAL

ENGENHARIA DE SOFTWARE PARA INTELIGÊNCIA ARTIFICIAL 0
NOSSA HISTÓRIA 3
1. INTRODUÇÃO 4
2. DEFINIÇÃO 4
3. HISTÓRICO 9
3.1 A Invenção de Turing 14
4. SOFTWARE E ENGENHARIA DE SOFTWARE 26
4.1 Software 27
4.2 Problemas associados ao software 28
4.3 A Importância do Software 28
4.4 O Papel evolutivo do Software 29
4.5 Aplicações do software 34
5. ENGENHARIA DE SOFTWARE: UMA DEFINIÇÃO 35
5.1 O que é engenharia de Software? 36
6. INTELIGÊNCIA ARTIFICIAL APLICADA NA ENGENHARIA DE SOFTWARE 39
7. AMBIENTES DE ENGENHARIA DE SOFTWARE CENTRADOS NO PROCESSO 42
8. TÉCNICAS DE INTELIGÊNCIA ARTIFICIAL 44
8.1 Sistemas Especialistas 45
8.2 Sistemas baseados em regras 45
8.3 Sistemas Fuzzy 46
8.4 Raciocínio baseado em casos 47
8.5 Agentes 48
8.6 Aprendizado 50
9. PSEES QUE UTILIZAM TÉCNICAS DE IA 50
10. AVALIAÇÃO E PROPOSTAS 53
10.1 Avaliação 54
10.2 Propostas 56
10.2.1 Modelagem e Reutilização 56

10.2.2 Execução 57

10.2.3 Simulação 57

10.2.4 Interface 58

11. INTELIGÊNCIA ARTIFICIAL NA ENGENHARIA DE SOFTWARE59

11.1 Redes neurais 59

12. REFERÊNCIAS: 64

NOSSA HISTÓRIA

A nossa história inicia-se com a ideia visionária e da realização do sonho de um grupo de empresários na busca de atender à crescente demanda de cursos de Graduação e Pós-Graduação. E assim foi criado o Instituto, como uma entidade capaz de oferecer serviços educacionais em nível superior.

O Instituto tem como objetivo formar cidadão nas diferentes áreas de conhecimento, aptos para a inserção em diversos setores profissionais e para a participação no desenvolvimento da sociedade brasileira, e assim, colaborar na sua formação continuada. Também promover a divulgação de conhecimentos científicos, técnicos e culturais, que constituem patrimônio da humanidade, transmitindo e propagando os saberes através do ensino, utilizando-se de publicações e/ou outras normas de comunicação.

Tem como missão oferecer qualidade de ensino, conhecimento e cultura, de forma confiável e eficiente, para que o aluno tenha oportunidade de construir uma base profissional e ética, primando sempre pela inovação tecnológica, excelência no atendimento e valor do serviço oferecido. E dessa forma, conquistar o espaço de uma das instituições modelo no país na oferta de cursos de qualidade.

1. INTRODUÇÃO

Desde o surgimento do termo inteligência artificial, ele tem gerado certa polêmica na comunidade científica. A primeira dificuldade vem da própria definição da inteligência como qualidade. O que define algo como inteligente? E de que forma a inteligência pode ser medida? O comportamento de um animal buscando alimento pode ser considerado inteligente? Um homem que joga xadrez é mais inteligente que um homem que cuida da lavoura? E uma máquina que reproduza um comportamento que, quando feito pelo homem este é considerado inteligente, pode ser considerada da mesma forma? Estas e outras perguntas ainda são controversas, cuja resposta depende muito da bibliografia e do ponto de vista utilizados.

Outro aspecto controverso é a área de conhecimento em que o campo inteligência artificial se enquadra. Filosofia? Matemática? Engenharia? Biologia? Ciência da Computação? Hoje em dia a resposta mais aceita é a de que o campo inteligência artificial é uma ciência multidisciplinar essencialmente parte da Ciência da Computação, uma vez que o computador foi escolhido como a máquina para se “implantar” inteligência. Trataremos desses temas quando abordarmos o histórico da inteligência artificial.

2. DEFINIÇÃO

O que hoje conhecemos como inteligência artificial é um ramo da ciência da computação ao mesmo tempo recente (o termo surgiu oficialmente em 1956) e antigo, cujos princípios baseiam-se em ideias filosóficas, científicas e tecnológicas herdadas de outras ciências, entre elas a lógica, que já existe há 23 séculos (BITTENCOURT, 2006). A dificuldade em se definir a IA se transforma na dificuldade em definir o significado do termo inteligência propriamente dito. Todos somos capazes de identificar se determinado comportamento é ou não inteligente quando parte de um humano ou um animal, entretanto, tal observação é mais complexa quando se trata de programas de

computador. O estudioso de Ciência da Computação Edsger Dijkstra cunhou uma frase polêmica a respeito da IA que dizia: “Perguntar se uma máquina pode pensar é o mesmo que perguntar se um submarino pode nadar”. Ou seja, para Dijkstra, o objetivo final é o que importa, e se este foi atingido por uma máquina, pode-se considerar que esta máquina é inteligente.

Luger (2004) define IA como o ramo da ciência da computação que se ocupa da automação do comportamento inteligente. Fernandes (2008) esclarece que é a parte da ciência da computação voltada para o desenvolvimento de sistemas de computadores inteligentes, isto é, sistemas que exibem características que estão associadas à inteligência no comportamento humano, como compreensão da linguagem, aprendizado, raciocínio, resolução de problemas, entre outros.

A expressão “Inteligência Artificial” soa de maneira assustadora, e talvez muitos de nós reagíssemos como o teólogo em sua indignação ao ver que uma máquina pode fazer aquilo que achamos ser uma exclusividade do gênero humano: pensar e agir racionalmente, executando tarefas para as quais se supõe que a inteligência seja necessária. Talvez muitos já tenham ouvido falar de projetos mirabolantes, como a construção de “cérebros eletrônicos” que seriam ligados na tomada e teriam “pensamentos” iguais aos nossos, ou até mesmo poderiam comunicar-se conosco, falando normalmente.

O que muitos de nós não sabem, entretanto, é que a ideia de se criar algo parecido com “máquinas pensantes” ou uma inteligência artificial paralela à nossa é hoje um projeto no qual trabalham cientistas de várias partes do mundo. Esses cientistas trabalham em várias áreas do conhecimento humano: linguística, psicologia, filosofia, ciência da computação etc. O que os reúne é, entretanto, uma característica comum: a ideia de que é possível criar “máquinas pensantes” e que o caminho para isso é o estudo e a elaboração de sofisticados programas de computador.

Para os pesquisadores da Inteligência Artificial (que daqui por diante abreviaremos IA) a mente humana funciona como um computador, e por isso o estudo dos programas computacionais é a chave para se compreender alguma

coisa acerca de nossas atividades mentais. Podemos construir programas que imitem nossa capacidade de raciocinar, de perceber o mundo e identificar objetos que estão à nossa volta, e até mesmo de falar e de compreender nossa linguagem.

É esta grande novidade da IA, que a distingue de ciências afins como a cibernética e a computação, englobando-as num projeto muito mais ambicioso: a produção de comportamento inteligente. Tarefas para as quais se requer alguma inteligência já são executadas por algumas máquinas de que dispomos e que utilizamos para telefonar ou mesmo para lavar roupa. Contudo, esse tipo de máquinas não tem interesse para a IA de que falaremos aqui, cuja preocupação é não só aliviar o trabalho humano, mas também desvendar alguma coisa acerca da natureza da nossa mente. Para isso é preciso que essas máquinas realizem tarefas que requerem inteligência, e de uma maneira muito similar e próxima do modo como nós, seres humanos, as realizamos. Assim, por exemplo, não basta simplesmente projetar e criar uma máquina de calcular (como nós a temos e usamos todos os dias, carregando-a no bolso) para dizermos que estamos fazendo IA. É preciso que essa máquina imite nossa atividade mental quando estamos fazendo uma operação aritmética.

Claro que esta imitação não poderá ser absolutamente perfeita. Afinal, somos construídos de matéria viva e até agora ainda não pudemos construir um mecanismo totalmente semelhante ao nosso cérebro, com suas células e as ligações nervosas que existem entre elas. Mas, assim como existem mecanismos que imitam nossa capacidade de andar, como por exemplo, o automóvel, da mesma maneira o teórico da IA propõe-se a criar mecanismos que, embora não sejam idênticos a nós, possam imitar nossas atividades mentais. A imitação que se pode obter é, portanto, apenas aproximada, e é por isso que em IA fala-se da elaboração de programas de computador que são modelos de nossa capacidade de raciocinar, de enxergar, de falar etc.

A IA como projeto efetivo só se tornou possível após o aparecimento dos computadores modernos, ou seja, após a Segunda Guerra Mundial (de 1945 em

diante). Até então havia dificuldades técnicas que precisavam ser superadas para que o projeto dessas máquinas mais modernas pudesse sair do papel.

Quando apareceu pela primeira vez uma máquina dita “pensante” – uma máquina dotada de um programa que demonstrava automaticamente teoremas de matemática -, o impacto sobre as ciências do homem foi tremendo. Subitamente a comunidade científica percebeu que uma verdadeira revolução havia se iniciado. Uma revolução com profundas influências na psicologia, na linguística e na filosofia. A ciência da computação deixava de ser uma disciplina puramente técnica, e suas realizações passaram a estender-se para outros campos. A ideia de estudar a mente humana à semelhança de um programa de computador parecia despontar como uma nova etapa para as ciências humanas.

Para a psicologia, a IA trouxe uma revolução, na medida em que o novo modelo apontava para uma alternativa à turbulência teórica que os psicólogos estavam atravessando. Havia muita discussão sobre a própria natureza do estudo a que se propunham os cientistas dessa área: se o seu estudo devia apenas se concentrar no comportamento dos organismos, ou se a psicologia deveria ser um estudo de nossas atividades mentais mediante um autoexame, através do qual fecharíamos os olhos e tentaríamos perscrutar o que se passa em nossa cabeça quando raciocinamos. Ambas possibilidades deixavam muito a desejar.

Para a linguística, a IA significava uma revolução: agora seria possível criar um programa de computador onde estivessem representadas as estruturas gramaticais das diversas línguas humanas. Se isto pudesse ser feito, um sonho muito antigo seria realizado: teríamos descoberto a raiz comum de todas as línguas humanas e uma máquina universal de tradução tornar-se-ia possível. Mas foi realmente sobre a filosofia que o impacto da IA foi maior: criar uma máquina pensante significa desafiar uma velha tradição que coloca o homem e sua capacidade racional como algo único e original do universo. Mais do que isto, criar uma máquina pensante significa dizer que o pensamento pode ser recriado artificialmente sem que para isto precisemos de algo como uma “alma” ou outra marca divina.

Algumas questões que tradicionalmente atormentaram os filósofos ao longo dos séculos passaram a receber um novo enfoque a partir da IA: por exemplo, o problema das relações entre a mente e o corpo, que se arrasta há milênios. Durante muitos anos os filósofos discutiram entre si se os nossos estados mentais (raciocínio, sonhos, imagens mentais etc.) seriam apenas manifestações de nossa atividade cerebral (materialismo) ou se eles não seriam reveladores da existência de algo imaterial como, por exemplo, uma alma imortal (dualismo). A IA oferece uma nova perspectiva para situarmos este problema para além das soluções existentes, que pendem seja para o materialismo seja para o dualismo. A recepção dada pelos filósofos ao novo modelo da mente humana proposta pela IA foi, entretanto, muito ambígua. Alguns filósofos viram na IA uma alternativa para a filosofia tradicional, que deixaria de ser apenas um conjunto de discussões acadêmicas às vezes consideradas estéreis e inconclusivas. As propostas filosóficas poderiam agora ser testadas em laboratórios, criando-se modelos computacionais para nossa maneira de raciocinar, de perceber o mundo e de formar pensamentos e ideias a partir dos objetos que estão à nossa volta. Esta seria a verdadeira filosofia “experimental” que muitos filósofos do passado gostariam de ter visto.

Outros filósofos, contudo, reagiram com a mesma indignação que parece ter sido experimentada pelo teólogo medieval em visita ao palácio real. Reduzir o ser humano e o pensamento às atividades de uma máquina seria uma proposta no mínimo ultrajante. Se o pensamento humano pode ou não ser mecanizado, como pretendem os teóricos da IA, é uma questão que ainda permanece em aberto. Tudo dependerá ainda de realizações futuras e de algum tipo de consenso a que os filósofos ainda hesitam em chegar. Para se ter uma noção mais precisa do que a IA propõe como programa de pesquisa, é preciso saber um pouco de sua história, de suas realizações até agora, e saber, em linhas gerais, como funciona um computador, o que para muitos é ainda um mistério.

3. HISTÓRICO

Fazer uma história precisa do desenvolvimento da IA não é tarefa fácil. Isto porque, embora seu aparecimento como disciplina científica só tenha ocorrido a partir da Segunda Guerra Mundial (1939-1945), a ideia de construir uma máquina pensante ou uma criatura artificial que imitasse as habilidades humanas é muito antiga. Os primeiros registros de criaturas artificiais com habilidades humanas têm uma forma mítica ou por vezes lendária, tornando difícil uma separação nítida entre imaginação e realidade. A ideia é de fato muito antiga, mas as condições técnicas para a sua realização são coisa recente. É esta confusão entre mito e realidade e, por vezes, a impossibilidade de distingui-los que faz com que a IA possa ser considerada uma disciplina com um extenso passado, mas com uma história relativamente curta.

Um dos episódios mais interessantes do passado mítico da Ia é a lenda do Golém. Joseph Golém era um homem artificial que teria sido criado no fim do século XVI por um rabino de Praga, na Tchecoslováquia, que resolvera construir uma criatura inteligente, capaz de espionar os inimigos dos judeus – então confinados no gueto de Praga. O Golém era de fato um ser inteligente, mas que um dia se revoltou contra seu criador, o qual então lhe tirou a inteligência e o devolveu ao mundo do inanimado. Alguns registros mais recentes mostram que nos séculos XVII e XVIII proliferam mais mitos e lendas acerca de criaturas artificiais. Fala-se de um flautista mecânico que teria sido capaz de tocar seu instrumento com grande perfeição, e que teria sido construído lá pelos fins do século XVII. Há registros também do célebre “pato de Vaucanson”, que teria sido construído por um artífice homônimo. A grande novidade dessa criatura teria sido sua capacidade de bater as asas, andar, grasnar, comer grãos e expeli-los após a digestão – uma perfeita imitação das funções biológicas.

A existência passada dessas criaturas artificiais até hoje não está definitivamente comprovada. Sabe-se apenas que seus projetos estão registrados em alguns museus da Europa e que sua arquitetura interna teria sido extremamente complexa. Os séculos XVII e XVIII conheceram também pela

primeira vez uma preocupação filosófica com algumas implicações teóricas envolvidas na construção dos primeiros autômatos de que se tem notícia. Descartes (1596-1650), filósofo racionalista do século XVII e oficialmente considerado o criador da filosofia moderna, expressou este tipo de preocupação em várias passagens de sua obra, argumentando que os autômatos, por mais bem construídos que fossem, jamais se igualariam aos seres humanos em termos de suas habilidades mentais. Isto porque os autômatos nunca viriam a ter uma alma imortal, igual à nossa, que lhes permitisse agir livremente e encadear sentenças de modo a expressar pensamentos como nós, humanos, o fazemos.

Mesmo que se construísse um autômato com cordas vocais e boca semelhantes às de um ser humano, ele jamais seria capaz de falar. No máximo, seria um repetidor de palavras, como um papagaio, mas isso não significa falar: significa apenas pronunciar palavras de uma maneira vazia, pois atrás destas não haveria pensamentos. Esse tipo de argumento formulado por Descartes foi ressuscitado por um filósofo norte-americano contemporâneo, John Searle, na forma de uma forte objeção às pretensões da IA. Descartes não foi o único a se preocupar com os problemas filosóficos envolvidos na construção de autômatos e com a discussão de suas possíveis habilidades mentais. Um século depois outro filósofo francês, La Mettrie (1709-1751), escreveu um livro criticando os pontos de vista de Descartes. Ele sustentava uma posição oposta à de Descartes, argumentando em favor da ideia de que o pensamento não é o resultado da atividade de uma alma imortal e imaterial que teríamos dentro de nós. As faculdades de pensar e de falar, dizia La Mettrie, aparecem naquelas criaturas que têm um organismo mais complexo, com um cérebro mais desenvolvido. Se os animais ou os autômatos ainda não desenvolveram essas faculdades, isso se deve apenas a imperfeições fisiológicas ou mecânicas dessas criaturas, cujos organismos ou mecanismos têm um grau de complexidade inferior ao nosso.

La Mettrie defendeu suas ideias com grande paixão, e quando hoje em dia lemos sua obra, que infelizmente ainda não foi traduzida para o português, temos a impressão de estarmos diante de um verdadeiro manifesto em favor da

IA. Nos seus trabalhos, este autor declara, entusiasticamente, que da mesma maneira que a humanidade teve, no começo dos tempos, um herói grego como Prometeu, que roubou o fogo dos deuses para dá-lo aos homens, um dia teríamos um segundo Prometeu que construiria um homem mecânico capaz de falar.

O século XIX não foi muito fértil no que diz respeito a discussões filosóficas acerca das habilidades e possibilidades dos autômatos. Tudo se passou como se momentaneamente o assunto tivesse sido esquecido e as preocupações tivessem se voltado para outros temas que então inquietavam a humanidade: a questão social e o agigantamento do industrialismo, que parecia prometer uma completa modificação na imagem do mundo. Foi somente na literatura – e na literatura do movimento romântico – que o tema do surgimento de criaturas artificiais parece ter sido lembrado. Nessa época é publicado o famoso romance Frankenstein, que explora o mito de um ser criado a partir de membros e órgãos de outras criaturas artificialmente reunidas. O sopro vital era substituído por um choque elétrico que fez com que a criatura adquirisse vida. Mas Frankenstein era um monstro. Um monstro que logo em seguida se revoltou contra seu criador.

Mas se o século XIX não produziu uma literatura onde o tema dos autômatos aparecia de forma explícita, nem por isso se pode dizer que nele faltaram descobertas que mais tarde seriam úteis para a IA. Invenções matemáticas e avanços tecnológicos, resultantes da descoberta da eletricidade, foram registrados – avanços que propiciariam, mais tarde, a construção dos modernos computadores. A IA só aparecerá no século XX, e num contexto bem diferente do que poderíamos ter imaginado. O advento da Segunda Guerra Mundial trouxe pressões decisivas para a comunidade científica dos países aliados.

Os bombardeios aéreos feitos pelos nazistas sobre as cidades europeias pressionaram o desenvolvimento de canhões antiaéreos dotados de um sistema de pontaria que corrigisse os eventuais desvios causados pelo deslocamento do alvo e do próprio canhão no momento do disparo. Esse tipo de mecanismo de

autocorreção começou a ser visto como uma incipiente imitação de um comportamento humano. Para um observador leigo, tudo se passava como se o comportamento do canhão, ao perseguir seu alvo com precisão, estivesse sendo guiado por propósitos ou intenções semelhantes ao do ser humano.

Paralelamente a estes desenvolvimentos na indústria bélica, o surgimento dos campos de concentração abriu uma nova possibilidade: a realização de experimentos e estudos do cérebro em seres humanos. Esses estudos eram feitos não só com cobaias humanas (prisioneiros de guerra) como também com soldados que tivessem sofrido lesões cerebrais durante os combates. Através desses estudos alguns cientistas tentavam aprender alguma coisa acerca dos princípios gerais do funcionamento do cérebro. Isso incluía a localização das áreas cerebrais responsáveis pelas diversas atividades humanas. Assim, por exemplo, para se tentar localizar a área cerebral que corresponderia à função da fala, removiam-se sucessivamente partes do cérebro do paciente, até que este se visse impossibilitado de falar.

No fim da Segunda Guerra Mundial, os cientistas já tinham registrado importantes invenções na área eletrônica, além de pesquisas sobre mecanismos que imitavam ações humanas e estudos sobre cérebro humano desenvolvidos por médicos e por psicólogos. Isso os levou a programarem um encontro nos Estados Unidos, onde pesquisadores dessas áreas apresentavam suas descobertas, numa primeira tentativa de reuni-las e compor algo parecido com uma ciência geral do funcionamento da mente humana. Esse encontro ficou conhecido como o Simpósio de Hixon, e aconteceu em 1948.

Quando os cientistas se reuniram no Simpósio de Hixon, já sabiam que a construção de um computador eletrônico tinha se tornado uma realidade. Naquela época, os escritos do matemático inglês Alan Turing, que continham os princípios de funcionalidade dos computadores modernos, começavam a sair nas revistas especializadas. Alan Turing era um jovem preocupado com questões matemáticas, cuja adolescência havia sido marcada pelo interesse por problemas filosóficos, como, por exemplo, as relações entre a alma e o corpo e a reencarnação. Quando se dedicou ao estudo de certas questões matemáticas,

ele provavelmente não sabia que suas descobertas tornariam possível a construção dos computadores modernos. Ele descobriu o princípio fundamental do funcionamento dessas máquinas – um princípio que até hoje norteia a construção de computadores, por mais sofisticados que sejam.

Os resultados do Simpósio de Hixon não teriam sido tão surpreendentes se não levassem, através de uma intuição verdadeiramente criadora, a se estabelecer uma analogia entre o cérebro humano e os computadores. Essa analogia certamente foi produto do encontro entre psicólogos, neurofisiólogos e engenheiros eletrônicos que perceberam que o modo como estão dispostas as células do nosso cérebro (neurônios), ligadas através de fios nervosos minúsculos, é semelhante ao circuito elétrico de um computador eletrônico. Estava aberto o caminho para se dizer que a mente humana pode ser imitada por um computador. Com isto nascia a nova disciplina, a Inteligência Artificial.

As décadas seguintes foram marcadas por novas invenções e descobertas surpreendentes. Na década de 50, dois cientistas americanos desenvolveram um programa de computador capaz de demonstrar automaticamente teoremas matemáticos. Esse programa foi chamado de “O Teórico da Lógica”, e sua inovação estava no fato de ele poder realmente gerar demonstrações de teoremas, e não simplesmente apresentá-las através de um artifício de memória. Os dois cientistas norte-americanos estavam realmente convencidos de que sua máquina era uma autêntica simulação do modo como os seres humanos resolvem seus problemas matemáticos, e escreveram vários artigos a esse respeito.

Mas a maior novidade ainda estava por vir. No final da década de 60, aparece um programa de computador capaz de imitar um psicanalista. Esse psicanalista mecânico, que foi chamado “DOCTOR”, era na verdade uma variação de um outro programa batizado de “ELIZA”. ELIZA foi um programa originalmente desenvolvido para simular diálogos e conversas. O programa analisava as frases e devolvia as respostas utilizando a máquina de escrever. O princípio de funcionamento desse tipo de programa era simples: a sentença enviada pelo parceiro humano era decomposta, e suas partes enviadas para um

script armazenado no interior do computador. O script era um conjunto de regras semelhantes àquelas que são dadas para um ator quando se requer que ele improvise acerca de um tema qualquer. ELIZA podia receber vários tipos de script, e, dependendo do conteúdo destes desenvolver conversas acerca de vários temas.

Quando Eliza trabalhava com um script especial chamado Doctor, ele se transformava num psicanalista mecânico. O script era cuidadosamente elaborado para que as respostas simulassem o comportamento verbal de um psicanalista ao receber um paciente pela primeira vez. Muitas pessoas, na época, chegaram a afirmar que ELIZA era apenas um truque de memória, e que suas respostas sempre obedeciam a padrões preestabelecidos. Se ELIZA é ou não um autêntico psicanalista artificial é uma questão que pode ser discutida. Qualquer que seja a conclusão, uma coisa, porém é certa: Eliza é uma imitação muito convincente do comportamento verbal de um psicanalista humano.

Por volta de 1970 foi inventado, no Massachussetts Institute of Technology (o famoso MIT), nos Estados Unidos, um sistema chamado “SHRDLU”. O Shrdlu simulava uma espécie de robô fechado num ambiente artificial onde estavam colocados blocos de madeira coloridos, do tipo dos usados em jogos infantis. O Shrdlu era capaz de obedecer a instruções e falar sobre a posição dos blocos que movia e de seus “braços”.

3.1 A Invenção de Turing

Para muitas pessoas o computador é ainda uma máquina misteriosa e às vezes assustadora. Quando olhamos para um microcomputador e deparamos com sua tela, seu teclado, e sua impressora, temos às vezes a sensação de estarmos diante de uma máquina mágica. Mesmo a grande maioria daqueles que o utilizam para realizar operações complexas muitas vezes não sabe o que ocorre dentro dessas máquinas. A situação é semelhante à de um motorista que dirige um carro sem, entretanto, saber os princípios de funcionamento do motor. O princípio de funcionamento dos computadores é relativamente simples, mas foram precisos anos para que se pudesse descobri-lo. Essa descoberta deveu-

se a Alan Turing (1912–1954), um matemático inglês que, apesar de ter sido brilhante na sua época, teve uma vida particularmente atribulada: não pertencia à aristocracia da Inglaterra, o que lhe criava dificuldades em certos meios acadêmicos, e., ademais, era homossexual, o que escandalizava a sociedade britânica. Embora tenha morrido prematuramente (ele se suicidou, provavelmente por motivos que tinham a ver com sua homossexualidade), Turing deixou uma vasta produção de trabalhos e invenções matemáticas.

Foi na tentativa de resolver um problema matemático muito complexo que estava sendo discutido na década de 30 que ele criou a chamada máquina de Turing. Para termos uma ideia do que seja uma máquina de Turing basta que imaginemos uma longa fita de papel com símbolos e marcas a intervalos regulares, formando pequenos quadrados. Imaginemos agora que podemos estipular uma espécie de marcador ou um ponto fixo em relação ao qual pudéssemos mover a fita de papel para a esquerda ou para a direita. A situação de que falamos pode ser representada assim:

	S1	E2 s2	D1 s3	R s4	A s5	

Suponhamos agora que o nosso marcador tenha também um dispositivo que permita reconhecer se num determinado quadrado há um símbolo ou ainda não, imprimir e apagar símbolos que aparecem na fita e ainda movê-la para a esquerda ou para a direita, dependendo do símbolo que aparece impresso. Na fita que aparece na figura, os quadrados têm dois tipos de símbolos: letras minúsculas e letras maiúsculas. Mover a fita para a esquerda ou para a direita (e num número determinado de quadrados) dependerá do símbolo em maiúsculas e é identificado pelo marcador. Além de mover a fita em determinadas direções, o símbolo em maiúsculas pode significar que o marcador deve imprimir ou apagar um símbolo num certo quadrado.

Em outras palavras, os símbolos A, B, C, D, E etc. representam as instruções que devem ser seguidas pela máquina, movendo a fita ou apagando os outros símbolos s1, s2, s3 etc. No caso da fita que representamos na figura

anterior, podemos convencionar que os símbolos E e D significam mover a fita para a esquerda ou para a direita, e que o número que está junto de E ou de D representa o número de casas que se quer que fita mova, seja numa direção ou outra. R significa “imprima o símbolo em minúsculas que está ao lado”, A significa “apague”, e assim por diante.

Vamos agora fazer uma outra suposição: a de que alguém queira usar uma máquina deste tipo para efetuar uma operação aritmética simples, como, por exemplo, uma soma. Para que isto possa ser feito, cada número terá de ser representado por uma sucessão de I, ocupando cada um deles um quadrado. Assim, o número 2 será representado por dois quadrados, sendo que em cada um deles deve figurar o símbolo I. O número 3 ocupará três quadrados, cada um com o símbolo I, e assim por diante. Na nossa máquina isto aparecerá assim:

	I	I	+	I	I	I	

Que tipo de instruções teremos de dar à máquina para que ela efetue a operação 2+3, isto é, para que ela venha a representar o número 5? Para isto temos de fazer com que ela obedeça às seguintes instruções:

a) Apague o sinal +.

b) Imprima o sinal I na mesma casa.

c) Mova a fita duas casas para a esquerda do marcador, isto é, puxe a fita duas casas para a direita.

d) Apague o símbolo I.

Quando efetuamos a última operação, a fita estará assim:

	I	I	I	I	I	

E isto corresponde à representação do número 5, isto é, à soma desejada. É desta maneira que a máquina de Turing procede para efetuar uma soma. Que

novidade há nisto? Aparentemente nenhuma. Ao contrário, parece que conseguimos complicar um processo simples, cotidiano. Contudo, as coisas não são bem assim. O que Turing inovou com a invenção de sua máquina foi à descoberta de uma espécie de princípio geral para a construção de computadores.

Este princípio geral tem como ponto de partida a noção matemática de procedimento efetivo. As instruções que damos para a máquina têm de ser executadas passo a passo, formando uma sucessão. Cada vez que uma instrução é executada, a máquina passa de um estado para outro. A mudança de um estado para outro corresponde a uma mudança de configuração. Para se mudar de uma configuração para outra existem certas instruções (como, por exemplo, no nosso caso: mova a fita para a direita, apague um símbolo etc.) que estabelecem exatamente aquilo que deve ser feito.

Quando existe esse tipo de receita que diz exatamente o que deve ser feito para se passar de um estado para outro num processo, temos um procedimento efetivo, ou seja, um conjunto finito de instruções não ambíguas que nos dizem o que fazer passo a passo, e que nos garantem a obtenção de um resultado no final. Ora, a descoberta de Turing consiste no fato de ele ter demonstrado, através da invenção de sua máquina, que toda e qualquer tarefa que possa ser representada na forma de um procedimento efetivo pode ser mecanizada, ou seja, pode ser realizada por um computador. Com sua invenção ele demonstrou ademais, que todo e qualquer tipo de computador pode, em última análise, ser reduzido a uma máquina de Turing, pois, embora os computadores possam diferir entre si quanto à sua finalidade e até mesmo ao material de que são compostos, eles podem ser imitados por sua máquina. E isso sem dúvida torna a máquina de Turing um verdadeiro princípio universal.

A máquina de Turing que descrevemos há pouco e que faz adições pode ser vista como uma imitação das várias maquinas de calcular de que dispomos. Na fita podemos representar os números (cada unidade será um quadrado com um símbolo), e há procedimentos efetivos para realizar operações aritméticas,

ou seja, as operações aritméticas podem ser descritas através de um conjunto preciso de instruções: puxe a fita para a direita, para a esquerda etc.

Podemos dizer que nesta máquina rudimentar a fita contém uma representação dos números, e que o conjunto de instruções corresponde ao programa da máquina, da mesma maneira que dizemos que os computadores têm um programa. Se temos uma máquina que efetua adições e subtrações (uma maneira de imaginar a subtração seria pelo processo de apagar uma certa quantidade de símbolos da fita), podemos dizer que em princípio, nossa máquina será capaz de efetuar qualquer operação aritmética. Pois afinal, o que são multiplicações senão repetições de somas, e divisões a repetição de subtração? Dividir 16 por 4 significa quantas vezes o número 4 pode ser subtraído de 16. Claro que no caso da nossa máquina com fita e marcador, o programa ficaria bastante complicado, e se os números fossem grandes, a fita teria de ser extraordinariamente longa.

Máquinas de calcular constituem um grande e primeiro passo para mecanizar parte de nossas atividades mentais. Claro que máquinas de calcular já existiam antes da invenção da máquina de Turing. Mas o que torna a invenção de Turing realmente interessante é a possibilidade de mecanizar tarefas executadas pela nossa mente, desde que elas possam ser representadas por símbolos e na forma de procedimentos efetivos. Imaginemos agora que em vez de trabalharmos com uma máquina rudimentar, com fita e marcador, tenhamos uma máquina bem mais sofisticada, mas cujo princípio seja o mesmo de uma máquina de Turing. Uma das diferenças seria que em vez de termos uma fita onde os quadrados teriam vários símbolos, teríamos apenas dois símbolos básicos, 0 e 1. Representar números e instruções na fita desse tipo de máquina se torna muito mais complicado: é preciso usar uma série de artifícios quando se dispõe de apenas dois símbolos. Mas certamente há aqui uma vantagem: se os símbolos a serem utilizados são apenas 0 e 1, podemos traçar uma correspondência entre estes e um circuito elétrico, com uma série de interruptores do tipo daqueles que usamos para apagar ou acender a luz de uma sala.

Nesses interruptores só há dois estados possíveis: quando eles estão ligados, passa a corrente, acende-se a lâmpada. Quando estão desligados a situação é inversa: não passa a corrente, a lâmpada fica apagada. Tudo se passa como se pudéssemos imaginar que estes estados de cada interruptor correspondessem aos símbolos que estão nos quadrados da nossa fita de papel: 0 quando não passa corrente, e 1 quando a corrente passa. Ora, é exatamente este princípio que nos permite chegar a algo como uma representação elétrica do pensamento.

Nossa máquina mais sofisticada não terá fita, mas um complexo circuito com interruptores. Em vez de ser operada por um movimento da fita de um lado para outro que apaga ou imprime símbolos, ela terá uma forma mais sofisticada de transmitir as instruções desejadas. Poderemos também conceber um tipo de "marcador" mais sofisticado. Tudo isso pode ser feito hoje em dia ocupando-se em espaço cada vez menor: os progressos da eletrônica permitem a construção de circuitos cada vez mais complexos e mais minúsculos. Foram estes progressos que permitiram reduzir tanto o tamanho dos circuitos e das máquinas que hoje em dia uma calculadora é tão pequena quanto um relógio de pulso.

Mas o que dissemos até agora serve apenas para mostrar como uma pequena parte de nossas atividades mentais – aquelas relacionadas com operações aritméticas ou matemáticas – pode ser mecanizada. Mas e quanto ao resto de nossos pensamentos? Nem todas as nossas atividades mentais são dirigidas para realizar operações com números, e é precisamente a possibilidade de se mecanizar este outro tipo de atividades que constitui a grande novidade introduzida pelos computadores modernos – que nada mais são do que complexas máquinas de Turing que operam apenas com os símbolos 0 e 1.

E como podemos representar outros tipos de pensamentos além de números usando apenas os símbolos 0 e1? Para isto os pesquisadores da IA e aqueles que começaram a construir computadores mais sofisticados precisaram, inicialmente, usar um artifício. O ponto de partida de tudo é a ideia de que nossos pensamentos são expressos em linguagem – não apenas em linguagem falada, mas em linguagem escrita. Ora a linguagem escrita nada mais é do que um

sistema de símbolos construído a partir dos elementos básicos que compõem nosso alfabeto. O que precisamos então é arranjar um meio de representar todas as letras do alfabeto em termos de 0 e 1.

Sabemos que a totalidade das letras do alfabeto que usamos mais os outros caracteres normalmente empregados por nós, tais como números, vírgulas, pontos, espaço entre palavras, sinais de adição, subtração, multiplicação etc. totalizam 256 caracteres. Se se acha este número muito grande, basta dar uma olhada no teclado de uma máquina de escrever elétrica para se ver que na realidade usamos muito mais caracteres para expressar informação do que as letras do alfabeto. Com estes caracteres podemos expressar praticamente todo e qualquer pensamento, contar história da Revolução Francesa, a história da filosofia, realizar operações matemáticas e até escrever um livro sobre IA.

Vamos agora supor que temos um baralho com 256 cartas e que em cada uma delas está impresso um dos 256 caracteres de que falamos. Alguém seleciona uma carta ao acaso e a entrega para mim, virada com a face impressa para baixo, de maneira que eu não possa ver o caractere que está impresso nela. Essa pessoa pede que eu diga qual é a caractere que está impresso na carta.

Uma maneira de adivinhar o caractere é pegar a lista com os 256 caracteres e ir perguntando, um por um, até eu saber qual é aquele que está na carta. Ora, este é um método muito trabalhoso e demorado, pois corremos o risco de ter que repetir a mesma questão 256 vezes.

Mas certamente existe uma outra saída para este problema: em vez de fazer 256 perguntas, posso começar perguntando: está o caractere da carta na primeira metade da lista? Qualquer que seja a resposta, teremos dividido a lista em duas partes com 128 caracteres cada uma – e numa dessas metades o caractere terá de estar. Tendo isolado uma das metades da lista onde o caractere em questão se encontra, posso repetir a estratégia mais de uma vez e dividir os 128 caracteres em duas metades de 64. Repito a questão e isolo uma lista com 64 caracteres. O processo deve ser repetido sucessivamente até que

tenhamos uma lista com apenas 2 caracteres e o caractere a ser identificado será necessariamente um deles. É fácil ver que com esta estratégia reduzi o número de perguntas a apenas 8, pois partindo de 256 caracteres, para se chegar a apenas 2 é preciso dividir a lista inicial oito vezes: na primeira vez obtivemos duas sublistas de 128 caracteres, na segunda vez duas sublistas de 64 caracteres, até que na oitava vez restarão apenas dois caracteres.

Foi utilizando esta estratégia que os pesquisadores da IA tornaram possível a representação de letras de nosso alfabeto e de outros caracteres que usamos para expressar informação em termos de apenas dois símbolos, 0 e 1. Se quisermos que isto seja feito pelo próprio computador, temos de fornecer-lhe a lista com os 256 caracteres e instruções para identificar um caractere qualquer, por exemplo, a letra a, dividindo a lista em metades e metades de metades. Quando o caractere não se encontra na primeira metade da lista dos 256 isto é representado por um 0. Caso contrário, ele será representado por 1. Após a repetição do processo por oito vezes sucessivas, teremos uma sequência de oito 0 e 1. A letra a, por exemplo, é representada neste processo por 0100001. Mas a sequência de 0 e 1 que correspondem a uma letra precisam, por sua vez, ser combinadas com outras sequências de 0 e 1 para formar palavras, o que nos dá uma sequência ainda maior. A representação de uma sentença no computador pode construir uma sequência extraordinariamente longa de 0 e 1.

A transformação de palavras e sentenças em sequências de 0 e 1 significa que uma representação elétrica de pensamentos, em termos de circuitos com interruptores abertos ou fechados, tornou-se possível. Isto ficou ainda mais fascinante quando se passou a projetar circuitos capazes de representar as várias maneiras através das quais podemos combinar sentenças – o que equivale a ter representações elétricas de algumas de nossas formas básicas de raciocinar.

Quando raciocinamos, o que passa pela nossa cabeça são proposições diversas. Uma proposição é um pensamento expresso nessa sentença. Por exemplo, quando dizemos “A tarde está bela”, isto é um pensamento expresso nessa sentença. Uma proposição tem sempre uma característica específica: ela

só pode ser verdadeira ou falsa. No caso da proposição “A tarde está bela”, temos meios de decidir se ela é verdadeira ou falsa: Podemos olhar pela janela ou mesmo ir dar um passeio. Raciocinar é formular proposições e encadeá-las. O modo de encadear proposições para formar raciocínios corretos tem sido objeto de estudo desde a Antiguidade, quando surgiu uma disciplina com esta finalidade, a lógica. A partir do século passado a lógica conheceu avanços substanciais. Um destes avanços constituiu em se estabelecer uma representação simbólica para as proposições. Passou-se a usar as letras A, B, C etc para designar proposições, e logo se fez um estudo das maneiras passíveis de encadeá-las.

Estudos preliminares apontaram para a existência de quatro maneiras básicas de combinar proposições: a conjunção, a disjunção, a implicação e a bi implicação. Podemos também negar uma proposição qualquer, o que neste sistema de representação será feito colocando-se o sinal ~ na sua frente. A negação de A será representada por ~ ª A conjunção de proposições significa ligá-las pela partícula e. A conjunção é simbolizada em lógica pelo sinal ^. Se tivermos duas proposições, A e B, a conjunção será representada assim: A^B. Se a proposição A for “O sol está brilhando” e a proposição B for “A tarde está bela”, A ^ B significará: “O sol está brilhando e a tarde está bela”. A disjunção significa ligar proposições pela partícula ou. Assim, “O carro está andando ou o carro está parado” é um exemplo de disjunção. A implicação é uma ligação do tipo: “Se está chovendo, então a terra está molhada”.

A lógica tratou não apenas do modo como podemos encadear proposições, como também estabeleceu regras para sabermos quando estas ligações resultariam em verdades ou em falsidades. Assim, suponhamos, por exemplo, que as proposições “O sol está brilhando” e “A tarde está bela” sejam falsas. Neste caso, a conjunção destas proposições também será falsa. Se a primeira proposição for falsa, mas a segunda for verdadeira, a conjunção será falsa. Todas as possíveis combinações para todos os tipos de proposições são previstas pela lógica. As combinações são agrupadas de acordo com o tipo de ligação entre as proposições, e formam aquilo que chamamos de “tabelas de verdade”.

Foi usando este tipo de estratégia que os pesquisadores da IA encontraram um caminho para construir circuitos que imitam nossa maneira de raciocinar. Da mesma forma que uma proposição só pode ter dois valores de verdade possíveis, ou seja, ser verdadeira ou falsa, os interruptores de um circuito também só podem ter duas posições: ou estão abertos (a corrente não passa) ou estão fechados (a corrente passa). É com base nesta analogia que podemos conceber um processo de representação elétrica dos raciocínios humanos.

O processo se inicia quando o programador coloca no computador as proposições que comporão um determinado raciocínio. Em seguida ele precisará informar ao computador o que se deseja fazer, ou seja, se se quer ligar as proposições através de uma conjunção, de uma disjunção ou implicação em outras palavras, é preciso fornecer ao computador as instruções que deverão ser seguidas. A partir do momento em que as instruções são fornecidas, um determinado circuito será acionado. Dependendo do tipo de instrução que é fornecido, o circuito que é acionado terá interruptores que permanecerão abertos ou fechados com a passagem de corrente elétrica. A construção desses circuitos e o que os interruptores neles fazem quando passa corrente elétrica obedecerá ao esquema das tabelas de verdade.

Assim, por exemplo, se se convencionou que quando existe passagem de corrente estamos representando uma proposição verdadeira, um circuito encarregado de fazer a conjunção de duas proposições verdadeiras deve ser construído de modo que a passagem de corrente elétrica faça com que seus interruptores se fechem e façam a corrente passar, permitindo que uma lâmpada se acenda ou outro sinal se manifeste. Se uma das proposições for falsa, um interruptor permanecerá aberto, a corrente não passará pelo circuito e a lâmpada não se acenderá. Neste caso saberemos que o resultado da conjunção de proposições não é verdadeiro.

Este processo de mecanização dos raciocínios pode se tornar extremamente complexo, sobretudo quando se tem várias proposições e várias maneiras de conectá-las entre si. Neste caso, várias instruções terão de ser

fornecidas, e muitos circuitos com interruptores com posições e características diferentes serão acionados até que se tenha um resultado final. Os computadores modernos contam ainda com uma série de recursos adicionais além dos circuitos de que falamos. Esses recursos incluem a memória, que permite à máquina estocar informações (proposições, expressões) e retirá-las desse registro sempre que for necessário. Nessas máquinas mais complexas, o fornecimento de instruções pode se tornar uma operação complicada, que deve ser feita em linguagens especiais ou linguagens de programação. Contudo, é preciso assinalar que mesmo máquinas mais sofisticadas obedecem ao mesmo princípio de que falamos aqui, ou seja, tudo é processado e reduzido a dois símbolos básicos, 0 e 1, a chamada linguagem de máquina, que indica fluxo de corrente ou a sua ausência.

Isto nos dá uma imagem da quantidade e da complexidade de operações que são executadas por um computador num intervalo de poucos segundos. Operações que levaríamos horas para executar podem ser executadas em poucos segundos, embora o modo como são efetuadas seja extremamente complexo. Hoje em dia, em muitas situações, querer competir com a velocidade de um computador seria tão ridículo como querer apostar corrida com um avião. Mas alguém ainda poderia dizer: sem dúvida, este processo de mecanização do pensamento tem se revelado eficiente, mas será que é este o modo como a mente opera para podermos dizer que o computador é um autêntico modelo do funcionamento da mente humana? Se não se puder responder afirmativamente a esta pergunta, não tem sentido falarmos de IA. Em outras palavras, o que se está questionando é se este procedimento de mecanização pode ser identificado com nossos processos de pensamento, o que nos permite dizer que a afirmação de que máquina pode pensar tem sentido.

Alguns teóricos tentaram responder a esta questão apontando para uma analogia entre os circuitos elétricos de um computador e o modo de funcionamento de nosso cérebro. Em nosso cérebro existem bilhões de células, os neurônios, que estão ligados entre si por minúsculos fios nervosos. Sabemos que em nosso cérebro passam correntes elétricas, pesar de estas terem uma voltagem baixíssima. Será que o modo como raciocinamos não é idêntico ao

modo como o computador o faz? Será que ao raciocinarmos não estão passando correntes elétricas entre os neurônios, que funcionariam como minúsculos interruptores?

Levantar este tipo de questão significa, em outras palavras, perguntar qual é a natureza daquilo que normalmente chamamos de pensar. Indagar o que seja aquilo que chamamos de pensamento pode nos levar a formular outras questões ainda mais inquietantes: terá cabimento afirmar que quando um circuito de um computador é acionado e nos fornece o resultado de uma soma, essa máquina está efetivamente pensando? Até que ponto será legítimo afirmar que essas máquinas constituem uma autêntica mecanização dos processos mentais humanos? Não seriam os computadores simples imitações ou simples truques que permitem realizar certas operações – máquinas às quais não poderíamos atribuir todas as características e propriedades de um ser humano?

Turing parece ter percebido as dificuldades que este tipo de questão envolve quando escreveu os artigos em que descrevia os princípios de sua máquina. Nesses artigos ele sustentou que não há nenhuma razão para se supor que uma máquina não possa pensar. Suas respostas a esse tipo de questão foram por vezes sarcásticas. Ele dizia que mesmo que para se dizer que uma criatura pensa seja preciso que esta tenha alma, não há nenhum impedimento para que Deus ponha alma nos computadores e não apenas nos seres humanos. Sustentar que Deus não poderia fazer isto seria o mesmo que querer limitar seus poderes. Deus dá almas a quem ele quiser. Mas alguém ainda poderia dizer que mesmo que aceitássemos essas afirmações de Turing – de que as máquinas pensam - poderíamos ainda colocar as seguintes questões: Como saberíamos se uma máquina pensa ou não? Não é o pensar um processo invisível? Que critérios devemos utilizar para podermos afirmar que uma máquina pensa ou não?

Turing respondeu a estas perguntas da seguinte maneira: suponhamos que construíssemos uma máquina e a disfarçássemos tão bem que ela tivesse a aparência de um ser humano. Suponhamos também que ao observar o comportamento desta máquina, constatássemos que ela faz tudo o que um ser

humano faz – respondendo a perguntas, fazendo operações aritméticas etc. - e que não tivéssemos meios de distinguir entre o comportamento desta máquina e o de um ser humano qualquer. Neste caso, pergunta

Turing, haveria alguma razão para não supor que esta máquina pensa, da mesma maneira que nós, seres humanos, o fazemos? Afinal de contas, qual o critério que usamos habitualmente para supor que nossos semelhantes pensam, a não ser a observação de seu comportamento e suas reações cotidianas? E, mesmo que uma máquina tão perfeita ainda não tenha sido construída hoje em dia, o que impede que no futuro nossa tecnologia avance tanto que ela se torne possível? Não se dizia no século passado que o homem jamais seria capaz de ir à Lua?

As afirmações de Turing geraram muitas polêmicas na sua época, principalmente entre os filósofos. Mas mesmo aqueles que discordaram de Turing, por achar que o jovem matemático inglês estava querendo humilhar a espécie humana ao sugerir que o pensamento não é um privilégio dado a nós pelo Criador, reconheceram mais tarde que a IA traz importantes contribuições para a filosofia. Mesmo que a mente humana não possa ser retratada numa máquina, o surgimento dos computadores abre novas perspectivas para a compreensão de nossas próprias atividades mentais, e traz sugestões interessantes para reavaliar a natureza de problemas filosóficos tradicionais.

4. SOFTWARE E ENGENHARIA DE SOFTWARE

No início da década de 1980, uma reportagem de primeira página da revista Business Week apregoava a seguinte manchete: "Software: A Nova Força Propulsora". O software amadurecera - tornara-se um tema de preocupação administrativa. Em meados da década de 1980, uma reportagem de capa da Fortune lamentava "Uma Crescente Defasagem de Software" e, ao final da década, a Business Week avisava os gerentes sobre a "Armadilha do Software - Automatizar ou Não?”. No começo da década de 1990, uma

reportagem especial da Newsweek perguntava: "Podemos Confiar em Nosso Software?" enquanto o Wall Street Journal relacionava as "dores de parto" de uma grande empresa de software com um artigo de primeira página intitulado "Criar Software Novo: Era Uma Tarefa Agonizante...". Essas manchetes, e muitas outras iguais a elas, eram o anuncio de uma nova compreensão da importância do software de computador - as oportunidades que ele oferece e os perigos que apresenta.

O software agora ultrapassou o hardware como a chave para o sucesso de muitos sistemas baseados em computador. Seja o computador usado para dirigir um negócio, controlar um produto ou capacitar um sistema, o software é um fator que diferencia. A inteireza e a oportunidade. das informações oferecidas pelo software (e bancos de dados relacionados) diferenciam uma empresa de suas concorrentes. O projeto e a capacidade de ser "amigável ao ser humano" (human-friendly) de um produto de software diferenciam-no dos produtos concorrentes que tenham função idêntica em outros aspectos. A inteligência e a função oferecidas pelo software muitas vezes diferenciam dois produtos de consumo ou industrias idênticas. E o software que pode fazer a diferença.

4.1 Software

Há 20 anos, menos de 1% do público poderia descrever de forma inteligível o que significa "software de computador". Hoje, a maioria dos profissionais bem como a maior parte do público, acham que entendem o que é software. Será que entendem?

Uma descrição de software num livro didático poderia assumir a seguinte forma: "Software é:

(1) instruções (programas de computador) que, quando executadas, produzem a função e o desempenho desejados;

(2) estruturas de dados que possibilitam que os programas manipulem adequadamente a informação; e

(3) documentos que descrevem a operação e o uso dos programas".

Não há dúvida de que outras definições, mais completas, poderiam ser oferecidas. Mas precisamos de algo mais que uma definição formal. O software é um elemento de sistema lógico, e não físico. Portanto, o software tem características que são consideravelmente diferentes das do hardware:

1. O software é desenvolvido e projetado por engenharia, não manufaturado no sentido clássico.

2. O software não se desgasta.

3. A maioria dos softwares é feita sob medida em vez de ser montada a partir de componentes existentes.

4.2 Problemas associados ao software

Existem um conjunto de problemas associados ao software que vários autores chamam de “crise do software”. Este conjunto de problemas não se limitam ao software que não funciona adequadamente, mas abrange também problemas associados a forma de desenvolvimento destes softwares, a forma como é efetuada a manutenção destes softwares, como atender a demanda por novos softwares e como desenvolver novos softwares cada vez mais rapidamente.

Os problemas que atingem o desenvolvimento podem ser descritos como:

• Estimativas de prazos e de custos que são frequentemente imprecisos;

• Produtividade dos profissionais da área que não tem acompanhado a demanda por novos serviços;

• A qualidade do software desenvolvido que é insuficiente.

4.3 A Importância do Software

Durante as três primeiras décadas da era do computador, o principal desafio era desenvolver um hardware que reduzisse o custo de processamento e armazenagem de dados. Ao longo da década de 1980, avanços na

microeletrônica resultaram em maior poder de computação a um custo cada vez mais baixo. Hoje, o problema é diferente. O principal desafio durante a década de 1990 é melhorar a qualidade (e reduzir o custo) de soluções baseadas em computador - soluções que são implementadas com software.

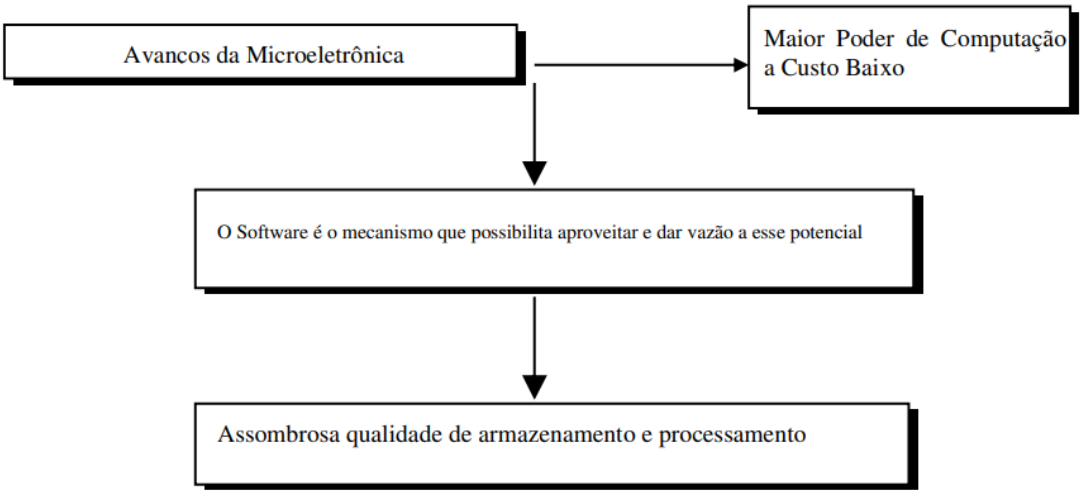

Figura:

O poder de um computador mainframe da década de 1980 agora está a disposição sobre uma mesa. As assombrosas capacidades de processamento e armazenagem do moderno hardware representam um grande potencial de computação. O software é o mecanismo que nos possibilita aproveitar e dar vazão a esse potencial.

4.4 O Papel evolutivo do Software

O contexto em que o software foi desenvolvido está estreitamente ligado a quase cinco décadas de evolução dos sistemas computadorizados. O melhor desempenho de hardware, menor tamanho e custo mais baixo precipitaram o aparecimento de sistemas baseados em computadores mais sofisticados. Mudamos dos processadores a válvula para os dispositivos microeletrônicos que são capazes de processar 200 milhões de instruções por segundo. Em livros populares sobre a "revolução do computador", Osborne caracterizou uma "nova revolução industrial", Toffler chamou o advento da microeletrônica de "a terceira onda de mudança" na história humana e Naisbitt previu que a transformação de

uma sociedade industrial em uma "sociedade de informação" terá um profundo impacto sobre nossas vidas. Feigenbaum e McCorduck sugeriram que a informação e o conhecimento (controlados por computador) serão o foco principal de poder no século XXI, e Stoll argumentou que a "comunidade eletrônica" criada por redes e software e a chave para a troca de conhecimentos em todo o mundo. Quando se iniciava a década de 1990, Toffler descreveu uma "mudança de poder", em que as velhas estruturas de poder (governamental, educacional, industrial, econômico e militar) se desintegrarão enquanto os computadores e o software levarão a uma "democratização do conhecimento".

Primeiros Anos 1950 a 1960	**Segunda Era 1960 a 1970**	**Terceira Era 1970 a 1980**	**Quarta Era 1980 a 2000**
Orientação Batch	Multiusuário Interativo	Sistemas Distribuídos	Sistemas de Desktop poderosos
Distribuição Limitada	Tempo Real	Hardware de Baixo Custo	Tecnologias Orientadas à Objetos
Software Customizado	Banco de Dados	Microprocessadores	Sistemas Especialistas
Programação Artesanal	Produto de Software	Impacto de Consumo	Computação paralela
Sem Administração Específica	Software House	"inteligência" embutida	Ferramentas CASE
Sem Documentação			Reutilização
			Redes Neurais artificiais

Tabela: 01

A tabela descreve a evolução do software dentro do contexto das áreas de aplicação de sistemas baseados em computador. Durante os primeiros anos do desenvolvimento de sistemas computadorizados, o hardware sofreu continuas mudanças, enquanto o software era visto por muitos como uma reflexão posterior. A programação de computador era uma arte "secundaria" para a qual havia poucos métodos sistemáticos. O desenvolvimento do software era feito virtualmente sem administração - até que os prazos começassem a se esgotar e os custos a subir abruptamente. Durante esse período, era usada uma orientação batch (em lote) para a maioria dos sistemas. Notáveis exceções foram os sistemas interativos, tais como o primeiro sistema de reservas da American Airlines e os sistemas de tempo real orientados a defesa, como o SAGE. Na maior parte, entretanto, o hardware dedicava-se a execução de um único programa que, por sua vez, dedicava-se a uma aplicação específica.

Durante os primeiros anos, o hardware de propósito geral tornara-se lugar comum. O software, por outro lado, era projetado sob medida para cada aplicação e tinha uma distribuição relativamente limitada. O produto software (isto e, programas desenvolvidos para serem vendidos a um ou mais clientes) estava em sua infância. A maior parte do software era desenvolvida e, em última análise, usada pela própria pessoa ou organização. Você escrevia-o, colocava-o em funcionamento e, se ele falhasse, era você quem o consertava. Uma vez que a rotatividade de empregos era baixa, os gerentes podiam dormir tranquilos com a certeza de que você estaria lá se defeitos fossem encontrados.

Por causa desse ambiente de software personalizado, o projeto era um processo implícito realizado no cérebro de alguém, e a documentação muitas vezes não existia. Durante os primeiros anos, aprendemos muito sobre a implementação de sistemas baseados em computador, mas relativamente pouco sobre engenharia de sistemas de computador. Por justiça, entretanto, devemos reconhecer os muito surpreendentes sistemas baseados em computador desenvolvidos durante essa época. Alguns deles permanecem em uso até hoje e constituem feitos que são um marco de referência e que continuam a justificar a admiração.

De 1960 até o final da década de 1970. A multiprogramação e os sistemas multiusuários introduziram novos conceitos de interação homem-máquina. As técnicas interativas abriram um novo mundo de aplicações e novos níveis de sofisticação de software e hardware. Sistemas de tempo real podiam coletar, analisar e transformar dados de múltiplas fontes, daí controlando processos e produzindo saída em milissegundos, e não em minutos. Os avanços da armazenagem on-line levaram à primeira geração de sistemas de gerenciamento de bancos de dados.

A segunda era também foi caracterizada pelo uso do produto de software e pelo advento das "software houses". O software era desenvolvido para ampla distribuição num mercado interdisciplinar. Programas para mainframes e minicomputadores eram distribuídos para centenas e, às vezes, milhares de

usuários. Empresários da indústria, governos e universidades puseram-se "desenvolver pacotes de software" e a ganhar muito dinheiro.

À medida que o número de sistemas baseados em computador crescia, bibliotecas de software de computador começaram a se expandir. Projetos de desenvolvimento internos nas empresas produziram dezenas de milhares de instruções de programa. Os produtos de software comprados no exterior acrescentaram centenas de milhares de novas instruções. Uma nuvem negra apareceu no horizonte. Todos esses programas - todas essas instruções - tinham de ser corrigidos quando eram detectadas falhas, alterados conforme as exigências do usuário se modificavam ou adaptados a um novo hardware que fosse comprado. Essas atividades foram chamadas coletivamente de manutenção de software. O esforço despendido na manutenção de software começou a absorver recursos em índices alarmantes.

E, ainda pior, a natureza personalizada de muitos programas tornava-os virtualmente impossíveis de sofrer manutenção. Uma "crise de software" agigantou-se no horizonte.

A terceira era da evolução dos sistemas computadorizados começou em meados da década de 1970 e continua até hoje. Os sistemas distribuídos - múltiplos computadores, cada um executando funções concorrentemente e comunicando-se um com o outro - aumentaram intensamente a complexidade dos sistemas baseados em computador. As redes globais e locais, as comunicações digitais de largura de banda (handwidth) elevada e a crescente demanda de acesso "instantâneo" a dados exigem muito dos desenvolvedores de software.

A terceira era também foi caracterizada pelo advento e generalização do uso de microprocessadores, computadores pessoais e poderosas estações de trabalho (workstations) de mesa. O microprocessador gerou um amplo conjunto de produtos inteligentes - de automóveis a fornos de microondas, de robôs industriais a equipamentos para diagnostico de soro sanguíneo. Em muitos casos, a tecnologia de software está sendo integrada a produtos por equipes

técnicas que entendem de hardware, mas que frequentemente são principiantes em desenvolvimento de software.

O computador pessoal foi o catalisador do crescimento de muitas empresas de software. Enquanto as empresas de software da segunda era vendiam centenas ou milhares de copias de seus programas, as empresas da terceira era vendem dezenas e até mesmo centenas de milhares de cópias. O hardware de computador pessoal está se tornando rapidamente um produto primário, enquanto o software oferece a característica capaz de diferenciar. De fato, quando a taxa de crescimento das vendas de computadores pessoais se estabilizou em meados da década de 1980, as vendas de software continuaram a crescer.

A quarta era do software de computador está apenas começando. As tecnologias orientadas a objetos estão rapidamente ocupando o lugar das abordagens mais convencionais para o desenvolvimento de software em muitas áreas de aplicação. Autores tais como Feigenhaum, McCorduck e Allman preveem que os computadores de "quinta geração", arquiteturas de computação radicalmente diferentes, e seu software correlato exercerão um profundo impacto sobre o equilíbrio do poder político e industrial em todo o mundo. As técnicas de "quarta geração" para o desenvolvimento de software já estão mudando a maneira segundo a qual alguns segmentos da comunidade de software constroem programas de computador. Os sistemas especialistas e o software de inteligência artificial finalmente saíram do laboratório para a aplicação pratica em problemas de amplo espectro do mundo real. O software de rede neural artificial abriu excitantes possibilidades para o reconhecimento de padrões e para capacidades de processamento de informações semelhantes as humanas.

Quando nos movimentamos para a quinta era, os problemas associados ao software de computador continuam a se intensificar:

• A sofisticação do software ultrapassou nossa capacidade de construir um software que extraia o potencial do hardware.

• Nossa capacidade de construir programas não pode acompanhar o ritmo da demanda de novos programas.

• Nossa capacidade de manter os programas existentes é ameaçada por projetos ruins e recursos inadequados.

Em resposta a esses problemas, estão sendo adotadas práticas de engenharia em toda a indústria.

4.5 Aplicações do software

O software pode ser aplicado a qualquer situação em que um conjunto previamente especificado de passos procedimentais (um algoritmo) tiver sido definido. O conteúdo de informações e a previsibilidade são fatores importantes na determinação da natureza de um aplicativo.

Desenvolver categorias genéricas para as aplicações de softwares é uma tarefa muito difícil. Quanto mais complexo é o sistema, mais difícil é determinar de forma clara os vários componentes do software.

Pode-se dividir as aplicações em:

• Software Básico – que é um conjunto de programas para dar apoio a outros programas. Tem como característica uma forte interação com o hardware, operações concorrentes, compartilhamento de recursos, uso por múltiplos usuários e múltiplas interfaces.

• Software de Tempo Real – são programas que monitora, analisa e controla eventos do mundo real, devendo responder aos estímulos do mundo externo com restrições de tempo pré-determinadas.

• Software Comercial – é a maior área de aplicação de softwares, são aplicações que gerenciam as operações comerciais de modo a facilitar o gerenciamento comercial do negócio da empresa, permitindo também a tomada de decisões.

• Software Cientifico e de Engenharia – são caracterizados por algoritmos de processamento numérico, dependentes da coleta e processamento de dados para as mais variadas áreas do conhecimento.

• Software Embutido – são desenvolvidos para executar atividades muito específicas e inseridos em produtos inteligentes tanto para atividades comerciais como para atividades domesticas.

• Software de Computador Pessoal – são produtos desenvolvidos para o uso pessoal do computador, tais como planilhas eletrônicas, processadores de textos, jogos, etc.

• Software de Inteligência Artificial – faz uso de algoritmos não-numéricos para resolver problemas complexos que não apresentam facilidades computacionais numéricas ou de análise direta.

5. ENGENHARIA DE SOFTWARE: UMA DEFINIÇÃO

Urna primeira definição de engenharia de software foi proposta por Fritz Bauer na primeira grande conferencia dedicada ao assunto: "O estabelecimento e uso de sólidos princípios de engenharia para que se possa obter economicamente um software que funcione eficientemente cm máquinas reais." A engenharia de software é uma derivação da engenharia de sistemas e de hardware. Ela abrange um conjunto de três elementos fundamentais - métodos, ferramentas e procedimentos - que possibilita ao gerente o controle do processo de desenvolvimento do software e oferece ao profissional uma base para a construção de software de alta qualidade produtivamente.

Os métodos de engenharia de software proporcionam os detalhes de "como fazer" para construir o software. Os métodos envolvem um amplo conjunto de tarefas que incluem: planejamento c estimativa de projeto, análise de requisitos de software e de sistemas, projeto da estrutura de dados, arquitetura de programa e algoritmo de processamento, codificação, teste e manutenção. Os métodos da engenharia de software muitas vezes introduzem uma notação gráfica ou orientada a linguagem especial e introduzem um conjunto de critérios para a qualidade do software.

As ferramentas de engenharia de software proporcionam apoio automatizado ou semiautomatizado aos métodos. Atualmente, existem ferramentas para sustentar cada um dos métodos anotados anteriormente. Quando as ferramentas são integradas de forma que a informação criada por uma ferramenta possa ser usada por outra, é estabelecido um sistema de suporte ao desenvolvimento de software chamado engenharia de software auxiliada por computador (CASE - Computer-Aided Software Engineering). O CASE combina software, hardware e um banco de dados de engenharia de software (uma estrutura de dados contendo importantes informações sobre analise, projeto, codificação e teste) para criar um ambiente de engenharia de software que seja análogo ao projeto auxiliado por computador/engenharia auxiliada por computador para o hardware.

Os procedimentos da engenharia de software constituem o elo de ligação que mantém juntos os métodos e as ferramentas e possibilita o desenvolvimento racional e oportuno do software de computador. Os procedimentos definem a sequência em que os métodos serão aplicados, os produtos (deliverables) que se exige que sejam entregues (documentos, relatórios, formulários etc.), os controles que ajudam a assegurar a qualidade e a coordenar as mudanças, e os marcos de referência que possibilitam aos gerentes de software avaliar o progresso.

A engenharia de software compreende um conjunto de etapas que envolve métodos, ferramentas e os procedimentos. Essas etapas muitas vezes são citadas como paradigmas da engenharia de software. Um paradigma de engenharia de software e escolhido tendo-se como base a natureza do projeto e da aplicação, os métodos e as ferramentas a serem usados, os controles e os produtos que precisam ser entregues.

5.1 O que é engenharia de Software?

É um conjunto integrado de métodos e ferramentas utilizadas para especificar, projetar, implementar e manter um sistema. Segundo Ariadne Carvalho & Thelma Chiossi no livro Introdução à Computação, a Engenharia de

Software é “Uma disciplina que reúne metodologias, métodos e ferramentas a ser utilizados, desde a percepção do problema até o momento em que o sistema desenvolvido deixa de ser operacional, visando resolve problemas inerentes ao processo de desenvolvimento e ao produto de software.” Pode-se definir como:

• Um método é uma prescrição explícita de como chegar a uma atividade requerida por um modelo de ciclo de vida, visando otimizar a execução das atividades que foram especificadas.

• As ferramentas proporcionam apoio automatizado ou semiautomatizado aos métodos.

Os Métodos de Desenvolvimento de Sistema se diferenciam pela maneira como o problema deve ser visualizado e como a solução do problema deve ser modelada. São eles:

Método baseado na decomposição de funções:

Abordagem estruturada caracterizada pela decomposição das funções. Os tipos de modelos que representam as funções são:

• DFD (Diagrama de Fluxo de Dados) – se caracteriza pela decomposição hierárquica de processos.

• MHT (Modelo Hierárquico de Tarefas) – se baseia na decomposição hierárquica de tarefas.

Método baseado na Estrutura de Dados:

Abordagem baseada na decomposição de um problema a partir dos dados. Exemplos de tipos de modelos dessa classe:

• MER (Modelagem Entidade-Relacionamento)

• Técnica de Warnier.

Método de análise baseado na orientação a objeto

Os tipos de modelos que representam essa classe são:

• UML (Unified Process) – notação de modelagem, independente de processos de desenvolvimento.

• Cenários

Paradigmas de engenharia de software

Segundo roger pressman, “paradigmas são os modelos de processos que possibilitam:

• ao gerente: o controle do processo de desenvolvimento de sistemas de software,

• ao desenvolvedor: a obter a base para produzir, de maneira eficiente, software que satisfaça os requisitos preestabelecidos.”

Os paradigmas especificam algumas atividades que devem ser executadas, assim como a ordem em que devem ser executadas. A função dos paradigmas é diminuir os problemas encontrados no processo de desenvolvimento do software, e deve ser escolhido de acordo com a natureza do projeto e do produto a ser desenvolvido, dos métodos e ferramenta a ser utilizado e dos controles e produtos intermediários desejados.

Processos de Software

Segundo Ian Sommerville, “Um processo de software é um conjunto de atividades e resultados associados que levam à produção de um produto de software.”

Embora existam muitos processos de software diferentes, há atividades fundamentais comuns a todos eles, tais como:

• Especificação de software;

• Projeto e implementação de software;

• Validação de software;

• Evolução do software.

A melhoria dos processos de software pode ser implementada de diferentes maneiras, como serão estudados posteriormente.

Os desafios da Engenharia de Software

Neste século XXI, os principais desafios serão:

• O desafio do legado, que é fazer a manutenção e a atualização dos softwares atuais, sem apresentar grandes custos e ao mesmo tempo prosseguir com a prestação dos serviços corporativos essenciais;

• O desafio da heterogeneidade, que se refere a desenvolver técnicas para construir softwares confiáveis e que sejam flexíveis para lidar com diferentes tipos de equipamentos e sistemas;

• O desafio do fornecimento, que deve apresentar uma redução do tempo gasto no desenvolvimento e implantação do software sem comprometer a qualidade.

6. INTELIGÊNCIA ARTIFICIAL APLICADA NA ENGENHARIA DE SOFTWARE

A computação tem evoluído muito nos últimos anos, e com ela o desenvolvimento de software. Várias técnicas e ferramentas foram desenvolvidas, porém a criação de produtos confiáveis e dentro do prazo ainda é muito pequena. Pesquisas apresentadas em 2009, pelo Standish Group, apontam que apenas 32% dos projetos de softwares desenvolvidos são concluídos com êxito. Estes problemas são comuns em empresas de desenvolvimento de softwares, pois analisar o risco durante o desenvolvimento, bem como o esforço necessário para o desenvolvimento não é uma tarefa fácil. Atualmente existem várias técnicas e métodos com o objetivo de solucionar estes problemas. Uma alternativa para estes problemas é a computação inteligente, que segundo Castro (2009), ferramentas inteligentes podem usar de históricos de projetos passados para contribuírem na estimativa de novos

projetos e também na avaliação de riscos. Neste sentido, Moura, Pinto e Lustosa Filho (2011), citam que sistemas inteligentes podem auxiliar ou mesmo substituir outros métodos.

A Engenharia de Software refere-se à aplicação disciplinada de princípios e métodos no projeto e construção de software de qualidade de forma economicamente viável. Alguns métodos procuram apoiar o gerente de desenvolvimento de software na tarefa de observar e controlar o desenvolvimento de software em termos de seus recursos, prazos, alocação de tarefas e orçamento. Entretanto, a natureza multidisciplinar da área demanda cooperação entre profissionais e ferramentas que apoiem o desenvolvimento de software.

Informalmente, o processo de desenvolvimento de software pode ser compreendido como o conjunto de todas as atividades necessárias para transformar os requisitos do usuário em software. O processo de desenvolvimento de software é formado por um conjunto de passos parcialmente ordenados, relacionados com conjuntos de artefatos, pessoas, recursos, estruturas organizacionais e restrições, tendo como objetivo produzir e manter os softwares requisitados. A partir de um esboço dos requisitos iniciais para o problema a ser resolvido, através de software, um modelo de processo de desenvolvimento de software será adotado, o qual resultará no software para atender os requisitos dos usuários.

A computação inteligente é a área da Ciência da Computação que busca a implementação de sistemas computacionais com base no comportamento humano, inspirados na capacidade de solucionar os problemas, bem como aprender com eles (CASTRO, 2009). Existem várias técnicas de inteligência artificial, porém neste trabalho serão destacadas as redes neurais, lógica fuzzy e as redes bayesianas. As redes neurais são sistemas compostos por unidades de processamento, chamadas de neurônios. Tais neurônios estão interligados por conexões e possuem um peso associado a eles, e com isso estes neurônios têm a capacidade de aprender, sendo este o diferencial desta técnica. (BORSOI et al; 2011). Outra técnica utilizada, a lógica fuzzy, conforme Boente (2009) é

uma teoria matemática que imita a capacidade humana de pensar em ambientes de incerteza, e assim, é possível o desenvolvimento de aplicações que trabalhem com valores imprecisos e subjetivos. Por último, as redes bayesianas também atuam sobre um mundo de incertezas, analisando a probabilidade de cada evento ocorrer (MENDES, 2010).

Com a tecnologia de processo de desenvolvimento de software, surgiu nos últimos anos um novo conceito de ambiente de trabalho que dá suporte de forma integrada aos processos de gestão e produção: Ambientes de Engenharia de Software Centrados no Processo, os chamados PSEEs (do inglês, Process-Centered Software Engineering Environments), que constituem um tipo especial de ambiente de desenvolvimento de software que apoia a definição rigorosa de processos de software, objetivando a automação da gerência do desenvolvimento. Tais ambientes geralmente proveem serviços para análise, simulação, execução e reutilização das definições de processos, que cooperam no aperfeiçoamento contínuo de processos.

Dentro deste ambiente, existem os agentes, que estão relacionados com as atividades de um processo, podendo ser pessoas ou ferramentas automatizadas. Agentes diferentes terão percepções diferentes acerca do que acontece durante o processo de desenvolvimento de software. Um gerente, por exemplo, perceberá os aspectos de controle e alocação de recursos e cronogramas para atividades, enquanto um desenvolvedor perceberá as suas atividades como atribuições que devem ser feitas para produzir um resultado.

Um SEE é definido como uma coleção de ferramentas que fornece apoio automático, parcial ou total, às atividades de Engenharia de Software. Normalmente essas atividades são executadas dentro de uma estrutura de projeto de software, e se referem a aspectos tais como especificação, desenvolvimento, reengenharia ou manutenção de software. O termo SEE pode ser aplicado a vários sistemas de alcances bem diferentes: desde um conjunto de poucas ferramentas executando sobre um mesmo sistema, até um ambiente totalmente integrado capaz de gerenciar e controlar todos os dados, processos e atividades do ciclo de vida de um software. Graças a automatização total ou

parcial das atividades, um SEE pode contribuir com importantes benefícios para uma organização: redução de custos, aumento da produtividade, melhoria da gestão e maior qualidade do software final. Por exemplo, automatização de atividade repetitivas como execução de casos de teste que não apenas melhoram a produtividade, mas também ajuda a garantir o término e a consistência das atividades testadas. Normalmente, um SEE gerencia informações relacionadas com:

• Desenvolvimento ou manutenção do software (especificações, dados de projeto, códigos fonte, dados de teste, planos de projeto, ...);

• Recursos do projeto (custos, recursos de informática, pessoal, responsabilidades e obrigações, ...);

• Aspectos organizacionais (política da organização, padrões e metodologias empregadas, ...).

Um SEE oferece suporte às atividades humanas mediante uma série de serviços que descrevem as capacidades do ambiente. Os serviços proporcionam uma correspondência entre um conjunto de processos selecionados, relativos ao ciclo de vida do software, e sua automatização mediante o uso de ferramentas. Na maioria dos casos as funcionalidades de uma ferramenta estão relacionadas com um ou mais serviços.

7. AMBIENTES DE ENGENHARIA DE SOFTWARE CENTRADOS NO PROCESSO

Os ambientes de Engenharia de Software centrados no processo, os chamados PSEEs (do inglês, Process-centred Software Engineering Environment) [16] constituem um tipo especial de ambientes de Engenharia de Software que surgiu nos últimos anos para apoiar a definição rigorosa de processos de software, objetivando automatizar a gerência do desenvolvimento.

Tais ambientes geralmente proveem serviços para análise, simulação, execução e reutilização das definições de processos, que cooperam no aperfeiçoamento contínuo de processos.

A modelagem de processos de software não consiste apenas em escrever programas que automatizem completamente o processo de desenvolvimento de software e nem descrevem tudo o que os atores do processo devem fazer. Enquanto os programas de computador são escritos para definir o comportamento de uma máquina determinística, os programas de processo são escritos para definir possíveis padrões de comportamento entre elementos não-determinísticos (atores) e ferramentas automatizadas. Como consequência, um PSEE deve ainda permitir que os atores envolvidos no processo recebam orientação automatizada e assistência na realização de suas atividades, sem interferência no processo criativo. Além disso, processos ainda podem ser modificados dinamicamente, em resposta a estímulos organizacionais ou mudança nos requisitos do software em desenvolvimento.

A literatura especializada define três tipos principais de modelos:

• Modelos Abstratos (patterns ou templates), que fornecem moldes de solução para um problema comum, em um nível de detalhe que idealmente não está associado a uma organização específica. Um processo abstrato é um modelo de alto nível que é projetado para regular a funcionalidade e interações entre os papéis de desenvolvedores, gerentes, usuários e ferramentas em um PSEE;

• Modelos Instanciados (ou executáveis) são modelos prontos para execução, podendo ser submetidos à execução por uma máquina de processo. O modelo instanciado é considerado uma instância de um modelo abstrato, com objetivos e restrições específicos, envolvendo agentes, prazos, orçamentos, recursos e um processo de desenvolvimento;

• Modelos em Execução ou Executados registram o passado histórico da execução de um processo, incluindo os eventos e modificações realizadas no modelo associado.

A arquitetura de um PSEE usualmente define como componente central a máquina de processo [6] que auxilia na coordenação das atividades realizadas por pessoas e por ferramentas automatizadas, sendo responsável pela interpretação/execução dos modelos de processos descritos com PMLs (Process Modelling Language). Uma máquina de processos é responsável por: ativar automaticamente atividades sem intervenção humana através de uma integração com as ferramentas do ambiente; apoiar o envolvimento cooperativo dos desenvolvedores; monitorar o andamento do processo e registrar o histórico da sua execução. A máquina de processo também deve garantir a execução das atividades na sequência definida no modelo de processo; a repetição de atividades; a informação de feedback sobre o andamento do processo; a gerência das informações de processo (incluindo gerência de versões); a coleta automática de métricas; a mudança do processo durante sua execução; a interação com as ferramentas do ambiente e a gerência de alocação de recursos.

O mecanismo de execução de processos de um SEE pode conter diversas instâncias simultâneas de máquinas de execução. Isto é necessário porque o SEE pode estar sendo utilizado para desenvolvimento em diversos projetos em uma organização. Portanto, de forma geral um mecanismo de execução consiste de uma ou várias máquinas de execução. As máquinas de execução possuem componentes que trabalham na execução de processos e na integração com o restante do ambiente pois a execução envolve desde a interface com o usuário até a gerência dos objetos no banco de dados. Dentro de um PSEE, o mecanismo de execução pode ser tratado como mais uma ferramenta ou pode ser um componente básico do ambiente.

8. TÉCNICAS DE INTELIGÊNCIA ARTIFICIAL

Inteligência Artificial é uma das ciências mais recentes, que atualmente abrange uma variedade enorme de subcampos, que vão desde áreas de uso geral, como aprendizado e percepção, até tarefas mais específicas, como jogos de xadrez [24]. A IA sistematiza e automatiza tarefas intelectuais e, portanto, é

potencialmente relevante para qualquer esfera da atividade intelectual humana, que neste caso, será abordada técnicas de IA para auxiliar o processo de desenvolvimento de softwares através dos PSEEs.

8.1 Sistemas Especialistas

Sistemas especialistas (SE's) são uma classe de software que atuam como colaboradores na tomada de decisão em áreas da ciência dominadas por especialistas humanos. Um sistema especialista condensa o conhecimento de um ou mais especialistas e utiliza este conhecimento armazenado para auxiliar na resolução de problemas do usuário. Os SE's são Sistemas Baseados em Conhecimento (SBC's) que atuam em áreas e em tarefas bem definidas. Estruturalmente, todo SE é constituído de duas partes principais: a Base de Conhecimento (BC), que contém o conhecimento heurístico e fatual sobre o domínio de aplicação do SE, e a Máquina de Inferência (MI), que usa o conhecimento da BC para construir a linha de raciocínio que leva à solução do problema.

O conhecimento obtido dos especialistas pode ser representado através de formalismos distintos: lógica, regras de produção, redes semânticas, frames e raciocínio baseado em casos. Embora tenham abordagens diferentes ao problema da representação do conhecimento, nada impede que sejam utilizadas combinações entre duas abordagens, como por exemplo, regras e frames para a representação do conhecimento de um determinado sistema.

8.2 Sistemas baseados em regras

O paradigma de regras é um dos paradigmas mais populares para representação do conhecimento em sistemas de IA. Conforme isso se deve, principalmente, à natureza modular das regras, as suas facilidades de explanação e por modelarem o conhecimento de uma forma muito próxima ao processo cognitivo humano. Uma regra consiste de uma parte SE, o lado esquerdo da regra, e de uma parte ENTÃO, o seu lado direito. A parte SE lista

um conjunto de condições combinadas de forma lógica e a parte ENTÃO representa a ação a ser executada ou a conclusão a ser deduzida, caso todas as condições da parte SE tenham sido satisfeitas. Sintaticamente, as regras podem ser representadas como:

$$\text{SE } \text{COND}_1, \text{COND}_2, \ldots \text{COND}_n \text{ ENTÃO}$$
$$\text{CONCL}_1, \ldots, \text{CONCL}_n$$
$$\text{OU}$$
$$\text{SE } \text{COND}_1, \text{COND}_2, \ldots \text{COND}_n \text{ ENTÃO}$$
$$\text{AÇÃO}_1, \ldots, \text{AÇÃO}_n$$

A coleção de predicados é chamada de Memória de Trabalho (MT). A parte ENTÃO da regra especifica novos predicados a serem colocados na MT. O sistema baseado em regras é dito ser um sistema dedutivo quando a parte ENTÃO somente contém predicados. Algumas vezes o ENTÃO especifica ações. Neste caso o sistema é dito reativo.

8.3 Sistemas Fuzzy

O interesse nos sistemas fuzzy tem aumentado nos últimos anos. A palavra fuzzy tornou-se adjetivo usado para descrever tudo o que não é absolutamente preciso, porém, nem sempre o termo é usado adequadamente. Devido ao uso indiscriminado deste termo, define sistemas fuzzy como: “Qualquer sistema computacional onde os valores não são precisamente predeterminados ou onde a confiança no sistema ou em seus dados possa ser discutida ou onde medidas de certeza estatísticas, probabilísticas ou ad-hoc sejam usadas para desenvolver parâmetros."

Os sistemas fuzzy combinam a flexibilidade e representação de conhecimento de alto nível dos sistemas especialistas convencionais com a habilidade de tratar problemas complexos e não lineares com o mínimo de regras.

Segundo os sistemas baseados em lógica fuzzy tem como principal benefício a redução dos custos de desenvolvimento, execução e manutenção.

Entretanto não são adequados para todos os tipos de sistemas em que os sistemas convencionais se aplicam. Diferente da lógica booleana, que possui os estados verdadeiro ou falso, a lógica fuzzy trata de valores verdade que variam continuamente de 0 a 1.

Dessa forma um fato pode ser meio verdade 0,5, quase verdade 0,9 ou quase falso 0.1. O uso da lógica fuzzy em sistemas de raciocínio traz impacto não somente na máquina de inferência, mas também na representação do conhecimento. A lógica fuzzy permite expressar conhecimento em um formato de regra que é bastante parecido com linguagem natural.

8.4 Raciocínio baseado em casos

Case-Based Reasoning (CBR) é uma técnica que utiliza a experiência passada para resolver problemas. A idéia de CBR é descrever e acumular casos significativos para a área de conhecimento especializado e tentar descobrir, por analogia, quando determinado problema é “similar “a um outro resolvido, aplicando a solução armazenada ao novo problema semelhante que surgiu. Um caso é um pedaço de conhecimento contextualizado representando uma experiência. Ele contém a solução do caso e o contexto onde essa solução pode ser usada. Um caso pode ser a descrição de um evento, uma história ou algum registro contendo tipicamente o problema no momento em que o caso ocorreu mais a solução para esse problema.

Uma maneira de visualizar um sistema de CBR é em termos de espaços de problemas e espaços de solução. A descrição de um novo problema a ser resolvido é posicionada no espaço do problema. O caso com a descrição mais similar é recuperado e sua solução é encontrada. Se necessário, pode ser feita uma adaptação e uma nova solução pode ser criada. Comparando os sistemas CBR com sistemas baseados em regras pode-se observar que para a criação de regras é sempre necessário saber como resolver o problema, sendo esta tarefa complexa e consumidora de muito tempo. Em sistemas CBR não é necessário saber como resolver os problemas, e sim apenas reconhecer se um problema similar foi resolvido no passado. Por isso, para problemas bem compreendidos

que não se modificam com o tempo, um sistema de regras é mais adequado. Porém, quando o problema é pouco conhecido e dinâmico, o CBR deve ser utilizado.

Apesar das diferenças, alguns sistemas híbridos (CBR + regras) têm sido desenvolvidos, tais como em [5], onde é apresentado um sistema para a área jurídica que recupera casos para auxiliar a denúncia de homicídios. Os sistemas CBR e os sistemas de redes neurais possuem como similaridade apenas o fato de se basearem em casos passados. As redes neurais são boas em domínios onde os dados não podem ser representados simbolicamente, tais como reconhecimento de voz e interpretação de sinais. CBR trata de forma adequada dados estruturados simbolicamente e complexos, mas não é tão bom para tratar dados puramente numéricos.

8.5 Agentes

Um agente é tudo o que se pode ser considerado capaz de perceber seu ambiente por meio de sensores e de agir sobre esse ambiente por meio de atuadores. Dado um determinado sistema, denomina-se agente cada uma de suas entidades ativas. O conjunto de agentes forma uma sociedade. As entidades passivas serão designadas pelo termo ambiente. Um agente raciocina sobre o ambiente, sobre os outros agentes e decide racionalmente quais objetivos deve perseguir, quais ações deve tomar, etc. O agente pode ser uma entidade real ou virtual que é capaz de agir em seu ambiente, podendo se comunicar com outros agentes, comportando-se de forma autônoma.

Os sistemas multiagentes dividem-se em duas classes principais: Agentes Reativos e Agentes Cognitivos. A abordagem reativa baseia-se na ideia de que agentes com ações elementares podem realizar trabalhos complexos. Segundo, nos sistemas multiagentes reativos não há representação explícita do conhecimento; não há representação do ambiente; não há memória das ações; a organização é etológica, ou seja, similar a dos animais; e existe grande número de membros.

Os sistemas multiagentes cognitivos são baseados em modelos organizacionais humanos, como grupos, hierarquias e mercados. Nestes sistemas os agentes mantêm uma representação explícita de seu ambiente e dos outros agentes da sociedade; podem manter um histórico das interações e ações passadas; a comunicação entre os agentes é direta, através de mensagens; seu mecanismo de controle é deliberativo, ou seja, os agentes raciocinam e decidem seus objetivos, planos e ações; seu modelo de organização é sociológico; uma sociedade contém poucos agentes.

Apesar da classificação, os sistemas multiagentes podem não ser totalmente cognitivos ou reativos. Um sistema pode ser uma mistura dos dois para atender a solução de um determinado problema. Um tipo de aplicação de agentes que pode também ser usado na Engenharia de Software são os agentes que reduzem o trabalho e o excesso de informação. Uma solução para o excesso de informação está no uso de Assistentes Pessoais, que são agentes autônomos capazes de atuar cooperativamente com o usuário para reduzir trabalho e excesso de informação. Um tipo de assistente pessoal é o agente de interface, o qual é capaz de realizar tarefas pelo usuário, treiná-lo, auxiliar na colaboração entre usuários e monitorar eventos. Os agentes de interface podem ser usados para filtragem de informações, gerência de correio eletrônico, agenda de compromissos, seleção de entretenimento, dentre outros.

As principais questões a serem tratadas são quanto a competência do agente, ou seja, como garantir que ele vai adquirir o conhecimento necessário; e a confiança do usuário, ou seja, como garantir que o usuário vai delegar tarefas para o agente. O agente adquire competência através da observação das ações do usuário, através do feedback do usuário quando o agente realiza uma ação, através de exemplos do usuário (treinamento do agente), pedindo conselho a outros agentes e aprendendo quais são os agentes mais confiáveis.

8.6 Aprendizado

O elemento central do comportamento inteligente é a habilidade de se adaptar ou aprender a partir de experiências. Existem várias técnicas de aprendizado, tais como:

• por implantação direta do conhecimento: através da inclusão de novas regras diretamente na base de conhecimento;

• por indução: perante um conjunto de exemplos ou dados particulares, o sistema procura inferir conceitos e leis gerais. Este aprendizado pode ser através de exemplos ou por observação e descoberta;

• por analogia: o sistema chega a conclusões sobre uma nova situação a partir de um modelo que foi construído como resultado de experiências anteriores;

• por casos: dados alguns casos de referência, para descobrir uma propriedade de uma situação ou dado particular, são encontrados os casos mais similares de acordo com as propriedades conhecidas;

• por construção de árvores de identificação;

• por redes neurais: Existem vários tipos de redes neurais. Elas podem ser usadas em aprendizado supervisionado (backpropagation) ou não supervisionado (mapa de Kohonen). Em uma rede de neurônios artificiais, existem somadores, multiplicadores e limites. Cada neurônio é disparado quando a influência coletiva de suas entradas atinge um limite mínimo. As entradas possuem pesos que são ajustados no processo de aprendizagem.

9. PSEES QUE UTILIZAM TÉCNICAS DE IA

A seguir são descritos brevemente alguns ambientes de Engenharia de Software que utilizam técnicas de Inteligência Artificial.

Marvel - Marvel é um ambiente centrado em processo resultante de um projeto de mesmo nome na Columbia University desde 1986. A ideia do projeto era desenvolver um ambiente centrado em processo que orientasse e assistisse os usuários que trabalham em projetos de grande escala. Para isso, juntaram as áreas de Engenharia de Software e Inteligência Artificial. Marvel segue o paradigma de orientação a objetos e está baseado em uma linguagem de regras. Ele fornece uma ajuda automatizada pela aplicação das estratégias forward e backward chaining sobre as regras, invocando as atividades que são parte de uma fase do processo de desenvolvimento. A sua base de objetos é persistente. O processo sendo modelado é decomposto em passos de processo. Cada passo é encapsulado em uma ou mais regras.

Merlin - Merlin é um ambiente de desenvolvimento de software centrado em processo construído dentro de um projeto na University of Dortmund. O protótipo do ambiente usa o paradigma de regras para descrever e executar processos de software.

Uma definição de processo no Merlin possui atividades, papéis (roles), documentos (quaisquer objetos) e recursos (pessoas, ferramentas, etc.). Um documento é ligado a um conjunto de atividades e a um conjunto de ferramentas que suportam as atividades. Nesta abordagem os usuários recebem todas as informações relevantes em um espaço de trabalho chamado working context associado ao papel do usuário. Este espaço de trabalho contém os documentos a serem manipulados, suas dependências com outros documentos e as atividades que devem ser realizadas. Quando uma atividade realizada influencia um working context, acontece uma atualização dinâmica nos working contexts dependentes deste evento.

A máquina de execução do Merlin constrói e atualiza os working contexts dos usuários. Cada atividade possui pré-condições que definem, por exemplo, os papéis que podem desempenhar a atividade, a pessoa responsável, os direitos de acesso aos documentos ou a dependência com outras atividades. Quando as precondições de uma atividade são verdadeiras, a atividade é inserida no working context. O processo de encadeamento é parecido com o do

ambiente Marvel apresentado anteriormente. O Merlin suporta mudanças dinâmicas no processo, permitindo que um processo não seja totalmente definido antes de iniciar. A linguagem escolhida para o ambiente é baseada em PROLOG.

Articulator - é um ambiente baseado em conhecimento para processo de desenvolvimento de software. Ele provê um meta-modelo de processo de desenvolvimento de software, uma linguagem baseada em objetos e um mecanismo de simulação automática. Para simular a execução de processos, o Articulator usa uma abordagem multiagentes onde os desenvolvedores são modelados como agentes cognitivos. A arquitetura do ambiente contém cinco subsistemas: base de conhecimento, simulador de comportamento, mecanismo de consulta, gerenciador de instanciação e gerenciador de aquisição de conhecimento.

O meta-modelo do Articulator consiste de recursos, agentes e tarefas. Os recursos são objetos nas tarefas dos agentes. As tarefas consomem e produzem recursos. Um agente representa uma coleção de comportamentos e atributos associados. O comportamento do agente emerge durante a execução das tarefas (incluindo comunicação, acomodação e negociação) levando em consideração a situação do agente. Os agentes são modelos gerais de desenvolvedores, times de desenvolvimento e organizações. Ferramentas de desenvolvimento são modeladas somo subclasse de agentes.

As tarefas são representadas através de uma rede de ações que os agentes realizam. Como não existe execução real de processos neste ambiente, os agentes são agentes cognitivos que simulam execução de tarefas e várias situações para auxiliar a geração da descrição do processo. A simulação é importante para detectar falhas como alocação de recursos, cronograma, dentre outras.

Pandora - Pandora é uma máquina de processos baseada em programação em lógica e conceitos de lógica temporal. Todos os eventos são registrados e o sistema possui um algoritmo de aplicação de regras que otimiza os passos de execução. Além disso, o sistema possui um mecanismo de

sincronização que garante que as atividades cooperativas ou que devem ser executadas em alguma ordem serão sincronizadas.

Pandora integra os paradigmas de eventos para modelar interatividade entre mensagens e de regras para representar o conhecimento do processo. O conhecimento pode ser declarativo ou procedural. Conhecimento declarativo descreve o domínio do discurso, no caso, o modelo de processo enquanto conhecimento procedural descreve o comportamento que o processo pode assumir durante o desenvolvimento, ou seja, as regras e eventos que disparam evolução de processo.

O sistema reage aos eventos externos e dispara as ações internas. As regras e eventos são especificados em uma linguagem de lógica de primeira ordem acrescida de operadores de lógica temporal (Linguagem Pandora) onde o tempo é caracterizado por uma linha sequencial simples de eventos. Isto permite expressar quantitativamente o tamanho dos intervalos temporais, a distância temporal entre os eventos e a viabilidade das atividades modeladas quanto às proposições lógicas estabelecidas. É possível estabelecer não apenas quais atividades são permitidas a cada momento, mas também como as atividades interagem (ou seja, sincronização entre atividades paralelas e disparo de atividades engatilhadas).

Pandora é implementado em PROLOG e consiste de dois módulos principais: compilador de regras e interpretador de regras. O compilador checa as regras quanto a sintaxe e as transforma em uma representação interna. O interpretador permite ativação das ferramentas do sistema operacional, permite rastreamento da aplicação de regras e observação dos estados internos.

10. AVALIAÇÃO E PROPOSTAS

Esta seção apresenta a avaliação do uso das técnicas de IA em PSEEs levando em consideração aspectos de modelagem, reutilização de processo, execução, simulação de processo, diagnóstico e recuperação de falhas na

execução, arquitetura do SEE e interface com os usuários do ambiente. Em seguida são apresentadas algumas propostas para utilização de técnicas de IA em PSEE visando facilitar o uso e aumentar a produtividade tanto do desenvolvedor quanto do gerente.

10.1 Avaliação

Os ambientes pesquisados permitem modelagem e execução de processos de software, sendo que somente o Articulator permite simulação. A adoção de notações gráficas para modelagem de processos foi observada no ambiente Merlin. O paradigma de regras como forma de representação do processo ou como modelo de execução foi totalmente adotado pela tecnologia de processos face à grande quantidade de ambientes existentes que utilizam o paradigma. As causas para essa adoção podem estar no requisito de modificação dinâmica durante execução do processo, sendo o paradigma de regras apontado como flexível a mudanças, ou ainda a necessidade de raciocínio baseado em conhecimento. O conhecimento sobre processos pode ser armazenado em uma base de conhecimento e regras podem ser utilizadas para obter novas informações ou auxiliar no acesso a esse conhecimento.

Os SEEs encontrados não se comportam como sistemas especialistas. O conhecimento sobre modelos de processo fica embutido no ambiente e os modelos construídos são desenvolvidos por projetistas de processo, que o fazem como um roteiro do desenvolvimento de software. Este roteiro pode ser modificado a qualquer momento. Portanto o conhecimento adquirido não provém de um especialista em processo de desenvolvimento de software visando resolver ou diagnosticar uma situação e sim de experiências em modelagem de processos.

O PSEE estudado que mais utiliza técnicas de IA é o Articulator. Além de ser baseado em regras para modelagem e execução do processo, ele utiliza o paradigma de multiagentes para modelar o comportamento dos agentes do processo. Como o ambiente permite somente simulação do processo, os agentes são modelados para agirem da mesma forma que agiriam agentes

humanos. Neste caso, o conhecimento a ser modelado é complexo e baseia-se em perícia, experiência, estilo de trabalho, dentre outras características dos agentes. A este ambiente foi integrada uma abordagem para diagnóstico, replanejamento e reescalonamento chamada Articulation. Esta abordagem é muito útil, pois o conhecimento sobre as falhas da execução dos processos vai sendo armazenado e gera heurísticas para a solução de novas falhas. Outra ferramenta incorporada ao Articulator é a biblioteca de processos SPLib. Ela permite reutilização de processos e possui uma base de conhecimento de processos que organiza os modelos e suas dependências. A forma de acesso a um processo desta biblioteca é similar à técnica case-based reasoning uma vez que o usuário deve fornecer algumas características desejáveis do processo e a biblioteca faz o matching dos processos mais adequados.

Pode-se observar que nem todas as técnicas de Inteligência Artificial são utilizadas pelos PSEEs, como redes neurais por exemplo. Isto se justifica porque o modelo de processo e sua execução necessitam de dados simbólicos e bem estruturados e o uso de redes neurais é mais adequado quando os dados não podem ser representados simbolicamente, como reconhecimento de voz por exemplo. Porém existe um caso de utilização de rees neurais para geração automática de modelos formais do comportamento de processos de software a partir da execução utilizando-se uma técnica chamada process discovery, que foi testada usando redes neurais, cadeias de Markov e um método algorítmico, mas o teste com redes neurais não foi satisfatório.

A seguir é apresentada uma tabela comparativa dos ambientes estudados.

Técnicas de IA / Ambientes	Marvel	Merlin	Articulator	Pandora
regras	**Sim**	**Sim**	**Sim**	**Sim**
case-based reasoning	Não	Não	**Sim**	Não
lógica *fuzzy*	Não	Não	Não	Não
sistemas multiagentes	Não	Não	**Sim**	Não
redes neurais	Não	Não	Não	Não
técnicas de aprendizado	Não	Não	**Sim**	Não

Tabela 1: Comparação do uso de técnicas de IA pelos ambientes centrados em processo

10.2 Propostas

As técnicas de IA ainda são muito pouco exploradas na Engenharia de Software. As principais utilizações se restringem ao paradigma de regras para modelar sistemas baseados em conhecimento que auxiliam o desenvolvimento de software. Este artigo procurou características de utilização de Inteligência Artificial em PSEEs. Foram apresentadas algumas técnicas de IA e alguns ambientes, com o objetivo de avaliar a utilização das técnicas nos ambientes. Entretanto o uso ainda é muito restrito, o que nos leva a propor algumas utilizações. Nas seções a seguir são apresentadas algumas ideias a respeito das técnicas de IA que podem ser utilizadas em modelagem, reutilização, execução, simulação e interface com o usuário de um PSEE.

10.2.1 Modelagem e Reutilização

Uma técnica de IA que aumentaria o poder de expressão de um modelo de processo seria a lógica fuzzy. Através de regras fuzzy, situações do tipo “se nível de qualidade da tarefa anterior é alto, então faça operação 1". Neste caso, o nível de qualidade depende de uma avaliação que não é precisa, mas que é importante para o desenvolvimento do processo. A reutilização de processos de software é altamente necessário e tem sido proposto em vários trabalhos. Para

a recuperação de processos de software adequados à situação que se quer modelar, podem ser usados sistemas de case-based reasoning como na biblioteca SPLib do sistema Articulator.

10.2.2 Execução

Na fase de execução do meta-processo de desenvolvimento de software, o componente principal é a máquina de processo. Este componente centraliza o controle do desenvolvimento de software e é responsável por inúmeras tarefas, como verificar consistência, direitos de acesso, próximas atividades a serem executadas, agendas dos usuários, estados das atividades, dentre outras. Para distribuir essas tarefas poderiam ser utilizados agentes da IA. Um agente poderia cuidar das agendas dos usuários, outro das atividades e seus estados e assim por diante. Uma vantagem dessa abordagem é a capacidade de aprendizado dos agentes, os quais poderiam agir de forma mais apropriada às peculiaridades do ambiente e seus usuários. Neste caso utilizaria sistemas multiagentes cognitivos. Os agentes reativos também poderiam ser usados para coletar métricas, por exemplo. Eles observariam o que acontece no ambiente e aumentariam a base de métricas com informações importantes sobre a execução do processo.

Há também a possibilidade de se utilizar lógica fuzzy na execução de processos. A máquina de execução, ao observar o atraso em uma atividade, pode detectar que o desenvolvedor que causou o atraso é “preguiçoso". Assim, pode existir uma regra fuzzy que verifica se a atividade está atrasada e o desenvolvedor é preguiçoso, então delegar automaticamente a tarefa para um agente mais responsável. A variável preguiçosa seria um conjunto fuzzy derivado de várias características. Case-based reasoning também seria aplicável na fase de execução pois em casos de parada na execução, o sistema poderia buscar um caso parecido e resolver o problema com uma solução testada.

10.2.3 Simulação

O SEE deve suportar validação e verificação de modelos de processo. Através da simulação de processos de software o ambiente e os usuários podem

detectar falhas, inconsistências e comportamentos anômalos na descrição do processo. Conflitos no cronograma e alocação de recursos são comuns. Além disso, o usuário pode prever o progresso do desenvolvimento e comparar modelos. Na simulação de processos não existe o desenvolvedor humano. Portanto é preciso simular o comportamento dele. A abordagem de agentes tem sido utilizada para essa finalidade como apresentado pelo ambiente Articulator. A simulação de processos pode usar também lógica fuzzy da mesma forma que foi proposta na execução.

Para que a simulação atinja seus objetivos de apontar os problemas da modelagem e da execução é necessário que ela seja baseada em situações reais. Portanto um sistema de case-based reasoning poderia ser usado para prover conhecimento real sobre situações de execução de processo e como são resolvidas essas situações. Na simulação também poderia se utilizar características reais dos agentes humanos para modelar os agentes cognitivos. Dessa forma, a simulação poderia auxiliar a resolver conflitos entre os desenvolvedores e a selecionar atividades para os mesmos.

10.2.4 Interface

A interface com o usuário de um PSEE depende do papel que ele executa no desenvolvimento. Um gerente precisa ver resultados quantitativos e qualitativos do trabalho dos desenvolvedores. As suas ações visam ajustar o processo para que não atrase e atinja os objetivos, sendo que para isso ele necessita interagir com os desenvolvedores. O desenvolvedor precisa ver as tarefas a serem desenvolvidas e os objetos a serem manipulados. Precisa também priorizar e delegar tarefas quando possível.

Os agentes de interface poderiam auxiliar esses usuários. Da mesma forma que um agente pode adquirir competência no tratamento das mensagens de seus usuários, ele pode adquirir competência no tratamento das tarefas que o desenvolvedor deve executar e tornar-se seu assistente pessoal de processo. O agente pode auxiliar um usuário novato indicando as tarefas mais importantes e fornecendo todas as informações necessárias para o desenvolvimento das

mesmas. Por outro lado, com um usuário mais experiente, o agente pode deduzir que o mesmo não necessita de tantas informações para realizar seu trabalho e apresentar somente as informações necessárias. Adquirindo competência, o agente pode automaticamente delegar tarefas para outros desenvolvedores, marcar reuniões e trocar informações com outros agentes para solucionar os problemas que surgirem.

Acredita-se que esta abordagem pode aumentar a utilização de PSEEs, pois um dos principais problemas da execução automatizada é a obrigação do seguimento de alguns passos do processo. Um desenvolvedor experiente não fica satisfeito com um sistema que o obriga a seguir passos programados e informa a todo instante como fazer seu trabalho. Um agente de interface pode identificar isso e filtrar as informações de acordo com o nível do usuário facilitando assim o aumento da produtividade no desenvolvimento de software.

11. INTELIGÊNCIA ARTIFICIAL NA ENGENHARIA DE SOFTWARE

11.1 Redes neurais

As redes neurais são desenvolvidas semelhantemente ao cérebro humano, sendo composta por vários neurônios artificiais. Estas redes são simuladas através de programação em computadores ou com a utilização de componentes eletrônicos (HAYKIN, 2001).

Os neurônios artificiais, também chamados de nós, são as unidades que compõem uma rede neural artificial. Estes neurônios recebem uma série de valores de entrada, juntamente com um peso, que determina a intensidade e o sinal da conexão. Cada neurônio possui uma função de ativação, que recebe a soma de todas as entradas multiplicadas pelos seus respectivos pesos, processa tais valores e gera uma saída para o neurônio (RUSSELL, NORVIG; 2004). Utilizando-se de redes neurais um trabalho desenvolvido foi de Moura, Pinto e Lustosa Filho (2011). Esta ferramenta foi desenvolvida para estimar os custos

de projetos de software da mesma forma como um engenheiro de software experiente estimaria.

Para efetuar o cálculo da estimativa, a ferramenta utilizará o modelo de pontos por função, levando-se em consideração pesos atribuídos a diversos fatores relevantes ao cálculo de estimativa. Com estes pesos são calculados os valores que servirão de entrada para a rede neural estimar o custo. Para a estimativa correta da ferramenta, a rede neural foi treinada com dados de 70 projetos reais já desenvolvidos, e os dados de outros 30 projetos foram utilizados para a validação dos valores estimados pela rede neural de acordo com seu aprendizado.

Outra ferramenta também foi desenvolvida utilizando redes neurais para a estimativa do custo de software, porém utilizando métodos diferentes para se chegar ao resultado. Neste trabalho, as entradas das redes neurais são os requisitos a serem desenvolvidos, bem como um tempo padrão para cada requisito ser implementado e o treinamento desta rede se dá a partir de tempos padrões indicados por especialistas.

Dada a execução desta ferramenta com vários conjuntos de tempos padrões e comparando a saída gerada pela ferramenta, é feito o treinamento da rede buscando o conhecimento necessário a ela, para que consiga estimar o custo de maneira semelhante a um especialista (BORSOI et al; 2011). Ainda utilizando redes neurais, outro trabalho foi encontrado com o objetivo de estimar o esforço final de um software, levando em consideração características semelhantes de todos os projetos. A rede foi desenvolvida utilizando três camadas, onde os valores de entrada correspondem à quantidade de características relevantes do problema, e a saída da rede será o custo estimado final do projeto. Para o treinamento da rede neural foram utilizados os dados de 214 projetos, sendo que parte destes foi utilizada para validação da rede (GARCIA et al; 2011).

Lógica Fuzzy

A lógica fuzzy é uma teoria matemática onde um objeto possui um grau de pertinência a uma determinada classe. Este grau de pertinência é um valor

entre zero e um que indica a adesão de um grupo a um conjunto (ZADEH, 1996). Para Boente (2009), é uma generalização do conceito de conjuntos, usados para representar um conjunto de informações imprecisas.

Em virtude das características desta técnica, ela foi usada em uma ferramenta para a avaliação da qualidade de produtos de software e a satisfação dos gerentes de projetos. Para o desenvolvimento da ferramenta foram seguidas algumas etapas. Entre estas etapas estavam as definições de termos e escalas que serão utilizados nas medições, como o nível de satisfação dos gerentes e o nível de qualidade presente no software. Feito isto, foram elaborados questionários para verificar a importância dos critérios de qualidade e satisfação dos gerentes e depois disto a formação das funções de pertinência. A partir disto foram aplicados os questionários e então coletados os dados e tratados, onde foi utilizada a fórmula da média fuzzy. Por fim os valores normalizados foram analisados e com isso foi obtido o índice da qualidade de software e satisfação dos gerentes de projeto (BOENTE, 2009).

Redes Bayesianas

A rede bayesiana segundo Mendes (2010):

> "é um modelo que permite a representação de um domínio complexo de conhecimento de uma forma que permite a manipulação desse conhecimento levando em consideração o grau de incerteza inerente a esse domínio. Tal incerteza deve idealmente ser considerada em todas as tomadas de decisão relativas a esse domínio do conhecimento".

Toda rede bayesiana é composta por duas partes, onde uma delas é a parte qualitativa, representada por um grafo, em que cada nó do grafo representa uma variável importante para a solução do problema proposto e as ligações entre estes nós representam de forma probabilística a relação entre as variáveis. A outra parte da rede, a parte quantitativa é representada pela tabela de probabilidades condicionais associada a cada nó, indicando as probabilidades inerentes ao nó (MENDES, 2010).

Esta técnica foi utilizada no desenvolvimento de uma ferramenta para estimativa de custo de aplicações web, utilizando o conhecimento de um especialista. A primeira etapa para o desenvolvimento foi identificar os

elementos que o especialista julga fundamentais para a estimativa do custo, usados como as variáveis da rede. Depois disto, o grafo foi montado, relacionando cada uma das variáveis identificadas, considerando a relação causa-efeito de cada uma. Com as variáveis e relações entre elas definidas, iniciou-se o processo do cálculo das probabilidades. A validação da ferramenta foi feita pelo especialista, executando vários cenários diferentes, onde todos os cenários foram executados com sucesso, além de outros testes, que foram feitos utilizando dados de projetos já desenvolvidos (MENDES, 2010).

Resumindo

As técnicas de Inteligência Artificial tem sido cada vez mais utilizadas em várias áreas da ciência da computação. Seu uso se justifica pelo auxílio fornecido ao raciocínio humano em tarefas que se beneficiam da experiência e do conhecimento de especialistas. Neste artigo foi apresentada a tecnologia de processos de software e suas características com o objetivo de encontrar utilização ou necessidade de utilização de técnicas de IA. Os PSEEs Marvel, Merlin, Articulator e Pandora foram estudados e avaliados. Com isso, podemos dizer que o ambiente que mais utiliza técnicas de IA dentre os apresentados é o Articulator, pois, permite simulação de processos e usa um modelo multiagentes para representar o comportamento dos desenvolvedores.

Também foram apresentadas algumas propostas de utilização de técnicas de IA em modelagem, execução, simulação, reutilização e interface do ambiente. As propostas foram derivadas do estudo das técnicas de Inteligência Artificial e do estudo da arquitetura e funcionamento dos PSEEs, sendo que essas propostas podem vir a ser implementadas em um futuro próximo, de forma a contribuir para o aumento da qualidade do software.

A utilização de técnicas de IA na Engenharia de Software ainda necessita de estudos e experiências, porém estão surgindo ambientes que se preocupam com os fatores psicológicos do desenvolvimento de software e sua facilidade de uso. As pesquisas na área apontam para o objetivo de tornar o PSEE um gerente de auxílio que não influencie a capacidade e a vontade dos desenvolvedores, mas ao mesmo tempo controle os recursos e os direitos de acesso, colete

métricas e disponibilize análises sobre o desempenho do processo de desenvolvimento de software. Acredita-se que as técnicas de IA ainda tem muito a contribuir para o desenvolvimento dessa área.

12. REFERÊNCIAS:

BOEHM, B. Software engineering. IEEE Transactions on Computers, December 1976.

BOENTE. Alfredo Nazareno Pereira. Um modelo fuzzy para avaliação da qualidade de produtos de software e da satisfação dos gerentes de projetos numa fundação pública estadual. 2009.

CASTRO. Raphael D'. Avaliação de riscos em projetos de software a partir do uso de técnicas de inteligência computacional. 2009.

COOK, J. and WOLF, A. Automating process discovery through event-data analysis. In INTERNATIONAL CONFERENCE ON SOFTWARE ENGINEERING, Seattle, Washington, United States, 1995.

COX, E. D. Fuzzy Logic for Business and Industry. Charles River Media, 1995.

DAHMER, A. Um modelo híbrido de sistema especialista para a Área jurídica. In CONGRESO ARGENTINO DE CIENCIAS DE LA COMPUTACIÓN, San Luis, Argentina, 1996.

DERNIAME, J., W., KABA, B. Software process: Principles, mehodology and technology. Springer, 1999.

DOWSON, M., R., NEJMEH, B. Fundamental software process concepts. AICA Press, May 1991.

FEILER, P. H., H. Software process development and enactment: Concepts and definitions. IEEE Computer Society Press, May 1993.

FRANCH, X., R. Supporting process reuse in promenade. February 2002.

G. JUNKERMANN, W. S., B. PEUSCHEL and WOLF, S. Merlin: Supporting cooperation in software development through a knowledge-based environment. In Software process modelling and technology archive, pages 103–129, London, UK, 1994. Research Studies Press Ltd.

GAIL E. KAISER, P. H. F. and POPOVICH, S. S. Intelligent assistance for software development and maintenance. JORNADA DE ATUALIZACAO EM INFORMATICA, pages 40–49, May 1988.

GIARRATANO, J. and RILEY, G. Expert Systems: Principles and Programming. PWS-KENT Publishing Company, 1989.

HUMPHREY, W. S. The software engineering process: Definitions and scope. IEEE Computer Society Press, 1989.

JUNKERMANN, G. and SCHÄFER, W. A design methodology for process-programming. In Proceedings of the Third European Workshop on Software Process Technology, pages 69–73, London, UK, February 1994. Springer-Verlag

LAGO, P. and G. MALNATI. Pandora: A temporal logic based process engine. WORKSHOP IN LOGIC PROGRAMMING APPLIED TO SOFTWARE ENGINEERING, 1994.

LIMA, C. Um gerenciador de processos de software para ambiente prosoft. Master's thesis, CPGCC-UFRGS, 1998. Dissertação de Mestrado.

LONCHAMP, J. A structured conceptual and terminological framework for the software process engineering. IEEE Computer Society Press, May 1993.

MAES, P. and KOZIEROK, R. Agents that reduce work and information overload. Communications of the ACM, 37(7), July 1994.

MENDES, Emília. Construindo um modelo especialista de estimativa de esforço de aplicações web usando redes bayesianas. 2010.

OSTERWEIL, L. Software process are software too. International Software Process Workshop, 1987.

REIS, C. A. L. Estudo da utilização de técnicas de inteligência artificial na tecnologia de processos de software. Trabalho Individual II - PPGC/UFRGS, março 1999.

REIS, R. Q., N. Uma avaliação dos paradigmas de linguagens de processo de software. Trabalho Individual II - PPGC/UFRGS, março 1999.

RUSSELL, S. and NORVIG, P. Artificial Intelligence: A Modern Approach. Prentice Hall, 2 edition, 2003.

SATLER, M. F. Utilidad de la tecnología de workflow en los psees. Dep. de Informática, Universidad de Castilla-La Mancha, 2004.

TULLY, C. Representing and enacting the software process. June 1989.

WANG, Y., K. Software engineering processes: Principles and applications. Boca Raton: CRC Press, 2000.

WATSON, I. Applying cade-based reasoning: Techniques for enterprise systems. CA: Morgan Kaufmann Publishers, 1997.